UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Software

Desarrollo de Aplicaciones Distribuidas por Capas

Guía teórico-práctica con IntelliJ IDEA, Java, Spring Boot y PostgreSQL

Asignatura	Aplicaciones Distribuidas (ISR-701)
Unidad	5: Desarrollo de aplicaciones en capas
Nivel	Séptimo nivel, Carrera de Software
Autor	Gleiston Cicerón Guerrero Ulloa, PhD
Correo	`gguerrero@uteq.edu.ec`
Versión	1.0, octubre de 2026

Herramientas en sus versiones vigentes al 26 de octubre de 2026: IntelliJ IDEA 2026.2, JDK 27, Spring Boot 4.1.2, PostgreSQL 18 y Docker Desktop 4.97.

Índice general

Presentación **iv**

1. Fundamentos de la arquitectura en capas **1**

1.1. Capas lógicas y niveles físicos 1

1.2. Las capas de una aplicación empresarial 1

1.3. La regla de dependencia 2

1.4. De las capas a los niveles: la arquitectura de BiblioUTEQ 2

1.5. Capas, niveles y microservicios 4

2. Principios y patrones de diseño **5**

2.1. Principios SOLID 5

2.2. Inversión de control e inyección de dependencias 5

2.3. Patrones de diseño aplicados 6

3. Entorno de desarrollo en Windows **8**

3.1. Versiones de las herramientas 8

3.2. Instalación paso a paso 9

3.3. Abrir y ejecutar el proyecto en IntelliJ IDEA 10

3.4. Crear un proyecto propio desde cero 10

4. Capa de datos y de acceso a datos **13**

4.1. El esquema como código: migraciones con Flyway 13

4.2. Entidades JPA 14

4.3. Repositorios con Spring Data JPA 15

4.4. Configuración del acceso a datos 16

4.5. Concurrencia optimista 17

5. Capa de negocio **19**

5.1. Reglas del negocio 19

5.2. El método fábrica del préstamo 19

5.3. El servicio de préstamos 20

5.4. Eventos del dominio: el patrón Observador 22

6. Capa de servicios: el API REST **24**

6.1. Diseño de los recursos 24

6.2. Objetos de transferencia de datos y validación 25
6.3. Los controladores 25
6.4. Errores con Problem Details (RFC 9457) 26
6.5. El contrato OpenAPI 27

7. Capa de presentación distribuida 29
7.1. Un cliente HTTP declarativo 29
7.2. Tiempos de espera: la primera defensa ante fallos parciales 29
7.3. Controladores web y el patrón Post/Redirect/Get 30
7.4. Cuando el API no responde 31
7.5. Las vistas 32

8. Integración y despliegue con contenedores 34
8.1. Imágenes de los niveles de aplicación 34
8.2. Orquestación local con Docker Compose 34
8.3. Levantar el sistema completo 35
8.4. Provocar una falla parcial 36

9. Pruebas de una aplicación distribuida en capas 38
9.1. Niveles de prueba en BiblioUTEQ 38
9.2. Pruebas unitarias de la capa de negocio 39
9.3. Pruebas de la capa de servicios web 39
9.4. Pruebas con una base de datos real: Testcontainers 40
9.5. Prueba de integración con concurrencia 42
9.6. Prueba de la capa de presentación 43

10. Calidad, integración continua y observabilidad 44
10.1. El modelo de calidad ISO/IEC 25010 44
10.2. Integración continua con GitHub Actions 45
10.3. Observabilidad 46

11. Aplicación al Proyecto Fin de Curso 47
11.1. Señales de que las capas se están erosionando 47
11.2. Actividades 47
11.3. Lista de verificación de una aplicación distribuida en capas 48

A. Estructura completa del proyecto 49

B. Comandos de referencia 50

Referencias **51**

Presentación

Esta guía acompaña la Unidad 5 de la asignatura Aplicaciones Distribuidas, cuyo resultado de aprendizaje es construir aplicaciones distribuidas en capas que cumplan estándares de calidad de software (ISO/IEC 25010 [1]), con seguridad y ética profesional, trabajando en equipo y en ciclos iterativos. La guía cubre los tres temas de la unidad: arquitectura en capas y patrones de diseño, desarrollo práctico de una aplicación distribuida en capas, y calidad, pruebas y despliegue.

La guía no es un resumen de teoría: se construye, capa por capa, una aplicación completa y funcional llamada **BiblioUTEQ**, un sistema de préstamos de libros para la biblioteca de la universidad. Cada concepto se explica primero en lenguaje sencillo, luego se nombra con su término técnico y, por último, se muestra en el código del proyecto. Todo el código que aparece en los listados se leyó directamente de los archivos del proyecto, que se compiló y se probó con las versiones de las herramientas indicadas en el capítulo 3 antes de publicar esta guía.

Cómo usar esta guía

1. Lea los capítulos 1 y 2 antes de escribir código: explican qué es una capa, qué es un nivel y por qué se separan.
2. Prepare su equipo con Windows siguiendo el capítulo 3. Al final de ese capítulo todas las herramientas deben responder a sus comandos de verificación.
3. Construya el proyecto en el orden de los capítulos 4 a 7: datos, negocio, servicios y presentación. Cada capítulo termina con una actividad práctica.
4. Despliegue el sistema completo con contenedores (capítulo 8), pruébelo (capítulo 9) y automatice su verificación (capítulo 10).
5. Aplique lo aprendido a su Proyecto Fin de Curso con las actividades del capítulo 11.

Convenciones

- Los comandos se escriben para **PowerShell en Windows 11**. Donde aparece `.\mvnw.cmd`, en Linux o macOS se usa `./mvnw`.
- El código fuente se escribe sin tildes ni eñes, para evitar problemas de codificación entre sistemas operativos. Los textos que ve el usuario final llevan sus tildes mediante entidades HTML o secuencias de escape Unicode.
- Las cajas azules amplían un concepto, las amarillas advierten de un error frecuente, las verdes resumen lo que debe quedar funcionando y las de borde turquesa proponen una práctica.

Material que acompaña a la guía

El proyecto completo BiblioUTEQ se entrega junto con esta guía (archivo `biblio-uteq.zip`). Contiene los dos módulos Maven, las migraciones de base de datos, las pruebas, los `Dockerfile`, el archivo de Docker Compose y el flujo de integración continua. Puede abrirse directamente en IntelliJ IDEA.

CAPÍTULO 1

Fundamentos de la arquitectura en capas

Una aplicación empresarial hace tres tipos de trabajo muy distintos: conversa con el usuario, decide qué está permitido hacer y guarda los datos para que no se pierdan. Cuando esos tres trabajos se mezclan en el mismo código, cualquier cambio pequeño (por ejemplo, cambiar la base de datos o agregar una aplicación móvil) obliga a revisar todo. La arquitectura en capas resuelve este problema separando el sistema en grupos de responsabilidades, cada uno con una sola razón para cambiar [2], [3].

1.1 Capas lógicas y niveles físicos

En español se usa la palabra «capa» para dos ideas que conviene distinguir desde el principio:

- Una **capa lógica** (en inglés, *layer*) es una forma de organizar el código: un grupo de clases con una responsabilidad común que solo puede usar los servicios de la capa inmediatamente inferior. Las capas lógicas pueden ejecutarse todas en el mismo proceso. En la práctica se ven como paquetes o módulos del proyecto.
- Un **nivel físico** (en inglés, *tier*) es un lugar de ejecución: un proceso, un contenedor o una máquina distinta, con la que las demás partes se comunican por la red [2], [4]. En la práctica se ven como programas que se arrancan por separado y se hablan por HTTP o por un protocolo de base de datos.

Una aplicación es **distribuida** cuando sus capas se reparten en varios niveles físicos que cooperan a través de la red. Esa distribución trae ventajas (cada nivel se escala, se actualiza y se protege por separado), pero también trae los problemas propios de los sistemas distribuidos: la red puede fallar o tardar, un nivel puede caerse mientras los otros siguen funcionando y varios usuarios pueden modificar el mismo dato al mismo tiempo [4], [5]. Esta guía trata esos problemas en el código, no solo en la teoría.

1.2 Las capas de una aplicación empresarial

La organización clásica distingue tres capas principales: presentación, dominio o negocio, y fuente de datos [2]. En las aplicaciones actuales que exponen un API, la capa de presentación se divide en dos: la interfaz de usuario y la capa de servicios web que publica el API [3]. La tabla 1.1 describe las capas que se usan en BiblioUTEQ.

Tabla 1.1: Capas de BiblioUTEQ y su responsabilidad

Capa	Responsabilidad (qué hace y qué no hace)	Dónde está en el proyecto
Presentación	Muestra la información y recoge las acciones del usuario. No decide reglas del negocio ni accede a la base de datos.	Módulo `biblio-web`: controladores web y plantillas Thymeleaf

Capa	Responsabilidad (qué hace y qué no hace)	Dónde está en el proyecto
Servicios web (API)	Traduce solicitudes HTTP en llamadas a la capa de negocio y sus resultados en respuestas HTTP. No contiene reglas del negocio.	Paquete `api.web` de `biblio-api`
Negocio	Ejecuta los casos de uso (prestar, devolver) y delimita las transacciones. No conoce HTTP ni SQL.	Paquete `api.negocio`
Dominio	Representa los conceptos del problema (libro, socio, préstamo) y protege sus reglas internas.	Paquete `api.dominio`
Acceso a datos	Guarda y recupera los objetos del dominio en la base de datos. No decide nada del negocio.	Paquete `api.datos`
Datos	Almacena los datos de forma persistente y hace cumplir las restricciones de integridad.	PostgreSQL 18 en su propio contenedor

1.3 La regla de dependencia

La regla que hace funcionar a las capas es simple: **cada capa solo puede usar a la capa que está debajo de ella, nunca a la de arriba**. Esta relación se conoce como «permitido usar» (*allowed-to-use*) [6]. Cuando además se prohíbe saltarse capas, las capas se llaman cerradas, y el beneficio es el aislamiento: un cambio dentro de una capa no afecta a las demás mientras su interfaz no cambie [3]. En la práctica, esto significa que en BiblioUTEQ ningún archivo del paquete `negocio` importa una clase del paquete `web`, y ningún controlador llama directamente a un repositorio.

La erosión de la arquitectura

Con el tiempo, los atajos (un controlador que consulta la base de datos «solo esta vez») hacen que la arquitectura implementada se aleje de la diseñada. Este fenómeno se llama erosión de la arquitectura y es una causa conocida de pérdida de mantenibilidad [7]. Se previene revisando las dependencias entre paquetes en cada revisión de código, como se propone en el capítulo 11.

1.4 De las capas a los niveles: la arquitectura de BiblioUTEQ

La figura 1.1 muestra cómo se reparten las capas de BiblioUTEQ en tres niveles físicos. El navegador del usuario solo habla con `biblio-web`; `biblio-web` solo habla con el API REST de `biblio-api`; y solo `biblio-api` habla con PostgreSQL. Esta disposición en tres niveles es una de las formas más extendidas de las arquitecturas cliente-servidor multinivel [4].

1.4.1 Ventajas y costos de distribuir

Distribuir las capas no es gratis. Separar la presentación en su propio nivel permite, por ejemplo, agregar una aplicación móvil que use el mismo API sin tocar la lógica del negocio, o escalar solo el nivel que recibe más carga. A cambio, cada llamada entre niveles atraviesa

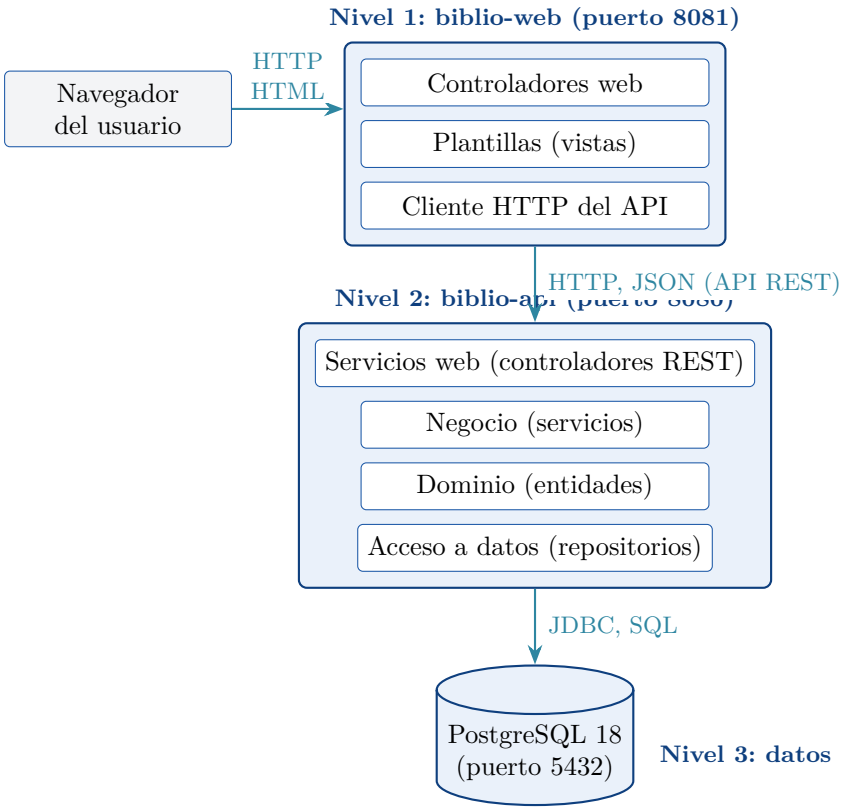


Figura 1.1: Capas lógicas de BiblioUTEQ repartidas en tres niveles físicos

la red, puede fallar por razones que no existen dentro de un solo proceso y exige decidir qué hacer cuando el otro nivel no responde [4]. La tabla 1.2 resume los compromisos (*trade-offs*) principales.

Tabla 1.2: Compromisos de separar capas en niveles físicos

Atributo	Lo que se gana	Lo que se paga
Escalabilidad	Cada nivel se replica según su propia carga	Hay que balancear la carga y mantener los niveles sin estado
Mantenibilidad	Cada nivel se cambia y despliega por separado	El contrato del API debe versionarse y cuidarse
Disponibilidad	La falla de un nivel puede aislarse	Aparecen fallas parciales que hay que manejar en el código
Rendimiento	Cada nivel usa los recursos que necesita	Cada llamada remota añade latencia de red y serialización
Seguridad	La base de datos no queda expuesta al usuario	Cada canal entre niveles debe protegerse

1.5 Capas, niveles y microservicios

La arquitectura en capas y la de microservicios no son excluyentes. Un microservicio bien diseñado suele estar organizado internamente en capas, y un sistema de microservicios es, visto desde fuera, un conjunto de niveles que se comunican por la red [8]. La diferencia está en cómo se corta el sistema: las capas lo cortan por tipo de responsabilidad técnica (presentación, negocio, datos), mientras que los microservicios lo cortan por capacidad del negocio (préstamos, catálogo, usuarios), cada una con su propia base de datos [3], [8]. La elección no es una cuestión de moda: los estudios comparativos muestran que la ventaja de los microservicios en rendimiento y escalabilidad depende del escenario de despliegue y de la carga [9]. BiblioUTEQ usa una distribución en tres niveles con capas internas, que es el punto de partida natural antes de dividir el sistema en microservicios.

Lo esencial de este capítulo

- Capa lógica es organización del código; nivel físico es lugar de ejecución.
- Cada capa usa solo a la inferior; los atajos erosionan la arquitectura.
- Distribuir capas en niveles da independencia a cambio de latencia y fallas parciales que el código debe manejar.

CAPÍTULO 2

Principios y patrones de diseño

Separar el código en capas no basta: dentro de cada capa las clases deben estar bien diseñadas para que el sistema sea fácil de cambiar y de probar. Este capítulo presenta los principios SOLID, la inversión de control con inyección de dependencias y los patrones de diseño que usa BiblioUTEQ, cada uno con el lugar exacto del proyecto donde se aplica.

2.1 Principios SOLID

SOLID es el nombre que agrupa cinco principios de diseño orientado a objetos cuyo objetivo es que los módulos toleren el cambio, sean fáciles de entender y puedan reutilizarse [10]. La tabla 2.1 los explica en lenguaje sencillo y muestra cómo se cumplen en el proyecto.

Tabla 2.1: Principios SOLID en BiblioUTEQ

Principio	Qué dice, en palabras sencillas	Cómo se ve en el proyecto
Responsabilidad única (SRP)	Una clase debe tener una sola razón para cambiar.	`PrestamoControlador` solo traduce HTTP; `PrestamoServicio` solo aplica el caso de uso.
Abierto/cerrado (OCP)	Se agrega comportamiento nuevo sin modificar el código que ya funciona.	Un nuevo observador de `PrestamoRegistradoEvento` se agrega sin tocar el servicio.
Sustitución de Liskov (LSP)	Un subtipo debe poder usarse donde se espera su tipo base sin romper nada.	`ManejadorErrores` extiende la clase base de Spring y conserva su contrato.
Segregación de interfaces (ISP)	Nadie debe depender de métodos que no usa.	Cada repositorio declara solo las consultas que su agregado necesita.
Inversión de dependencias (DIP)	Las políticas de alto nivel dependen de abstracciones, no de detalles.	El servicio depende de la interfaz `LibroRepositorio`, no de JPA ni de PostgreSQL.

2.2 Inversión de control e inyección de dependencias

En un programa tradicional, cada clase crea los objetos que necesita con `new`. Con la **inversión de control** (*Inversion of Control*, IoC) esa responsabilidad pasa a un contenedor: la clase solo declara lo que necesita y el contenedor se lo entrega ya construido. La forma de entregarlo

se llama **inyección de dependencias** (*Dependency Injection*, DI) y aplica directamente el principio de inversión de dependencias [10]. En Spring, el contenedor IoC crea los objetos marcados con `@Service`, `@Repository`, `@Component` o `@Configuration` y los entrega por el constructor.

El listado 2.1 muestra la inyección por constructor en `CatalogoServicio`. El servicio no sabe qué clase implementa `LibroRepositorio`: Spring Data la genera al arrancar. En una prueba unitaria, en cambio, se le entrega un objeto simulado, como se verá en el capítulo 9.

Listado 2.1: Inyección de dependencias por constructor en `CatalogoServicio`

```java
/** Capa de negocio: casos de uso del catalogo de libros y de los socios. */
@Service
@Transactional(readOnly = true)
public class CatalogoServicio {

    private final LibroRepositorio libros;
    private final SocioRepositorio socios;

    // Inyeccion de dependencias por constructor: Spring entrega las implementaciones.
    public CatalogoServicio(LibroRepositorio libros, SocioRepositorio socios) {
        this.libros = libros;
        this.socios = socios;
    }

    public List<Libro> listarLibros(String texto) {
```

Antipatrones de inyección de dependencias

La inyección de dependencias mal usada también degrada el diseño. Un estudio empírico catalogó antipatrones frecuentes, como inyectar clases concretas en lugar de abstracciones o acumular demasiadas dependencias en una sola clase [11]. En la práctica: prefiera la inyección por constructor (los campos quedan `final` y la clase no se puede crear incompleta), y si un constructor recibe más de cinco o seis dependencias, la clase probablemente tiene más de una responsabilidad.

2.3 Patrones de diseño aplicados

Un patrón de diseño es una solución probada a un problema que se repite, descrita de modo que pueda reutilizarse en contextos distintos [12]. La tabla 2.2 reúne los patrones que aparecen en BiblioUTEQ. Algunos los escribe el programador; otros los aplica el propio marco de trabajo y conviene reconocerlos.

Tabla 2.2: Patrones de diseño en BiblioUTEQ

Patrón	Problema que resuelve	Dónde se aplica
Repositorio (*Repository*) [2], [13]	Dar a la capa de negocio una colección de objetos del dominio sin que conozca el SQL.	`LibroRepositorio`, `PrestamoRepositorio` (capítulo 4)
Capa de servicio (*Service Layer*) [2]	Definir los casos de uso de la aplicación y sus límites transaccionales en un solo lugar.	`PrestamoServicio`, `CatalogoServicio` (capítulo 5)

Patrón	Problema que resuelve	Dónde se aplica
Modelo de dominio (*Domain Model*) [2], [13]	Mantener las reglas que pertenecen a un objeto dentro del propio objeto.	`Libro.retirarEjemplar()`, `Prestamo.registrarDevolucion()`
Método fábrica (*Factory Method*) [12]	Crear objetos siempre en un estado válido, con un nombre que expresa la intención.	`Prestamo.registrar(...)`
Objeto de transferencia de datos (DTO) [2]	Transportar datos entre niveles sin exponer las entidades internas.	Registros `LibroRespuesta`, `PrestamoSolicitud` (capítulo 6)
Proxy [12]	Interponer un sustituto que controla el acceso al objeto real.	Las transacciones de `@Transactional` y el cliente HTTP declarativo `BiblioApiCliente` (capítulo 7)
Observador (*Observer*) [12]	Notificar a interesados sin que el emisor los conozca.	`PrestamoRegistradoEvento` y `NotificadorPrestamos`

2.3.1 El patrón Proxy que no se ve

Cuando una clase tiene la anotación `@Transactional`, Spring no entrega a los demás objetos la instancia real, sino un **proxy**: un objeto que se hace pasar por ella, abre la transacción antes de llamar al método real y la confirma o la revierte al terminar [12]. Por eso, si un método de un servicio llama a otro método `@Transactional` de la misma clase, la segunda anotación no tiene efecto: la llamada no pasa por el proxy. En la práctica, los límites transaccionales se definen en los métodos públicos que llaman los controladores, como se hace en BiblioUTEQ.

Práctica 2.1: reconocer patrones

Abra el proyecto en IntelliJ IDEA y, para cada fila de la tabla 2.2, ubique la clase indicada con *Navigate* → *Class* (`Ctrl+N`). Escriba en una frase qué pasaría en el sistema si ese patrón no se hubiera aplicado. Por ejemplo: sin el método fábrica, otra clase podría crear un préstamo sin retirar el ejemplar del inventario.

CAPÍTULO 3

Entorno de desarrollo en Windows

Este capítulo prepara un equipo con Windows 11 para desarrollar, ejecutar y probar BiblioUTEQ. Se usan las versiones vigentes de cada herramienta al 26 de octubre de 2026, verificadas en sus fuentes oficiales.

3.1 Versiones de las herramientas

Tabla 3.1: Herramientas y versiones usadas en la guía

Herramienta	Versión	Observación
IntelliJ IDEA	2026.2.3 (compilación 262.10968.63)	Desde la versión 2025.3 es un solo producto con funciones gratuitas y una suscripción Ultimate [14]; la 2026.2 da soporte a Java 27 desde el primer día [15].
JDK (Eclipse Temurin)	27	Disponible desde el 15 de septiembre de 2026 [16]; la versión de soporte extendido (LTS) más reciente es la 25 [17].
Spring Boot	4.1.2	Publicada el 23 de octubre de 2026 [18]; requiere Java 17 o superior y es compatible hasta Java 27 [19].
PostgreSQL	18.6	Versión estable más reciente [20]; la 19 está en fase de candidata y su publicación está prevista para el 19 de noviembre de 2026 [21].
Docker Desktop	4.97.0	Publicada el 26 de octubre de 2026 [22]; incluye Docker Compose v2.
Maven	3.9.16	Se usa a través de Maven Wrapper (`mvnw.cmd`); no hace falta instalarlo. Maven 4.0 todavía está en fase de candidata.
Git para Windows	La más reciente	Para clonar el proyecto y versionar el trabajo.

Las bibliotecas que el proyecto usa dentro de Spring Boot no se eligen a mano: Spring Boot 4.1.2 fija versiones compatibles entre sí. Las más relevantes para esta guía son Spring Framework 7.0.10, Hibernate ORM 7.4.13, Flyway 12.4.0, el controlador JDBC de PostgreSQL 42.7.14, Testcontainers 2.0.5, JUnit Jupiter 6.0.3 y Tomcat 11.0.27. Para la documentación del API se agrega springdoc-openapi 3.2.0, la versión compatible con Spring Boot 4.1.2.

¿JDK 27 o JDK 25?

JDK 27 es la versión más reciente, pero no es de soporte extendido: recibirá actualizaciones hasta marzo de 2027, cuando la reemplace JDK 28 [16]. JDK 25 es la versión LTS más

reciente [17]. Esta guía usa JDK 27 porque el curso trabaja con lo más actual; para un sistema que deba mantenerse en producción varios años conviene JDK 25. El proyecto no usa funciones exclusivas de Java 27, así que para cambiar de versión basta con editar la propiedad `java.version` del `pom.xml` raíz.

3.2 Instalación paso a paso

3.2.1 JDK 27

1. Descargue el instalador MSI de Eclipse Temurin 27 para Windows x64 desde `https://adoptium.net/temurin/releases/`.
2. Durante la instalación, active las opciones *Add to PATH* y *Set JAVA_HOME variable*.
3. Abra una ventana nueva de PowerShell y verifique:

```
java -version
echo $env:JAVA_HOME
```

La primera línea de la respuesta debe indicar la versión 27, y `JAVA_HOME` debe apuntar a la carpeta del JDK 27.

3.2.2 IntelliJ IDEA 2026.2

1. Descargue el instalador desde `https://www.jetbrains.com/idea/download/`. Desde la versión 2025.3 hay un único instalador: las funciones principales para Java son gratuitas y las avanzadas forman parte de la suscripción Ultimate [14].
2. El asistente para crear proyectos Spring Boot y las herramientas básicas de base de datos son gratuitos; el soporte completo de Spring forma parte de Ultimate. La suscripción Ultimate es gratuita para estudiantes y docentes que la solicitan con su correo institucional en `https://www.jetbrains.com/shop/eform/students`.
3. Al abrir IntelliJ IDEA por primera vez, configure el JDK en *File* → *Project Structure* → *SDK*, eligiendo la carpeta del JDK 27.

3.2.3 Docker Desktop

1. Descargue Docker Desktop desde `https://www.docker.com/products/docker-desktop/` e instálelo con el motor basado en WSL 2 (*Windows Subsystem for Linux*), que es la opción recomendada en Windows 11.
2. Reinicie el equipo si el instalador lo pide y abra Docker Desktop. Debe quedar en estado *Engine running*.
3. Verifique en PowerShell:

```
docker version
docker compose version
docker run --rm postgres:18 postgres --version
```

La última orden descarga la imagen oficial de PostgreSQL 18 y debe responder `postgres (PostgreSQL) 18.6`.

3.2.4 PostgreSQL 18

En esta guía PostgreSQL se ejecuta en un contenedor, de modo que no hay que instalarlo en Windows: el archivo `compose.yaml` del proyecto lo levanta con la configuración correcta (capítulo 8). Si prefiere una instalación nativa, use el instalador de `https://www.postgresql.org/download/windows/`, cree la base de datos `biblio` con el usuario `biblio` y ajuste la variable `DB_URL` si usa otro puerto.

3.2.5 Git

Instale Git para Windows desde `https://git-scm.com/download/win` y configure su identidad, que es la que aparecerá en cada *commit*:

```
git config --global user.name "Nombres Apellidos"
git config --global user.email "usuario@uteq.edu.ec"
```

3.3 Abrir y ejecutar el proyecto en IntelliJ IDEA

1. Descomprima `biblio-uteq.zip` en una carpeta sin espacios ni tildes, por ejemplo `C:\Proyectos\biblio`.
2. En IntelliJ IDEA elija *File* → *Open*, seleccione el archivo `pom.xml` de la raíz y luego *Open as Project*. IntelliJ IDEA reconoce los dos módulos Maven, `biblio-api` y `biblio-web`, y descarga las dependencias.
3. Levante solo la base de datos desde la terminal integrada (*View* → *Tool Windows* → *Terminal*):

```
docker compose up -d db
```

4. Abra la clase `BiblioApiApplication` y pulse el triángulo verde junto a `main`. Al arrancar, Flyway crea las tablas y carga los datos iniciales; compruébelo en `http://localhost:8080/api/v1/libros`.
5. Abra la clase `BiblioWebApplication` y ejecútela del mismo modo. La aplicación web queda en `http://localhost:8081`.

Ver la base de datos desde IntelliJ IDEA

En la ventana *Database* pulse + → *Data Source* → *PostgreSQL* e ingrese `localhost`, puerto `5432`, base de datos `biblio`, usuario `biblio` y clave `biblio`. Podrá ver las tablas `libro`, `socio`, `prestamo` y la tabla `flyway_schema_history`, donde Flyway registra las migraciones aplicadas.

3.4 Crear un proyecto propio desde cero

Para su Proyecto Fin de Curso puede partir del asistente de IntelliJ IDEA: *File* → *New* → *Project* → *Spring Boot*. Elija lenguaje Java, tipo Maven, grupo `ec.edu.uteq`, JDK 27 y Spring Boot 4.1.2, y agregue las dependencias de la tabla 3.2. Si el asistente no ofrece Java 27 en la lista de versiones del lenguaje, elija la más alta disponible y cambie después la propiedad `java.version` del `pom.xml` a 27.

Tabla 3.2: Dependencias de cada módulo

Dependencia	Para qué se usa
biblio-api	
`spring-boot-starter-webmvc`	Servidor HTTP y controladores REST
`spring-boot-starter-validation`	Validación de los datos de entrada
`spring-boot-starter-data-jpa`	Acceso a datos con JPA e Hibernate
`spring-boot-starter-flyway` y `flyway-database-postgresql`	Migraciones versionadas del esquema
`postgresql`	Controlador JDBC de PostgreSQL
`spring-boot-starter-actuator`	Salud y métricas del servicio
`springdoc-openapi-starter-webmvc-ui`	Contrato OpenAPI y Swagger UI
`spring-boot-testcontainers`, `testcontainers-postgresql`	PostgreSQL real en las pruebas
biblio-web	
`spring-boot-starter-webmvc`	Servidor HTTP y controladores web
`spring-boot-starter-thymeleaf`	Plantillas HTML del lado del servidor
`spring-boot-starter-restclient`	Cliente HTTP para consumir el API

El listado 3.1 muestra el `pom.xml` raíz: hereda de `spring-boot-starter-parent` 4.1.2, que fija las versiones de todas las bibliotecas, y declara los dos módulos.

Listado 3.1: `pom.xml` raíz del proyecto

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.2</version>
        <relativePath/>
    </parent>

    <groupId>ec.edu.uteq</groupId>
    <artifactId>biblio</artifactId>
    <version>1.0.0</version>
    <packaging>pom</packaging>
    <name>biblio</name>
    <description>BiblioUTEQ: aplicacion distribuida en capas (guia ISR-701)</description>

    <modules>
        <module>biblio-api</module>
        <module>biblio-web</module>
```

```
    </modules>

    <properties>
        <java.version>27</java.version>
        <springdoc.version>3.2.0</springdoc.version>
    </properties>
```

Lo que debe quedar funcionando

`java -version` responde 27; `docker compose version` responde v2; IntelliJ IDEA abre el proyecto sin errores; `http://localhost:8080/api/v1/libros` devuelve cuatro libros en JSON y `http://localhost:8081` muestra el catálogo.

CAPÍTULO 4

Capa de datos y de acceso a datos

La capa de datos guarda la información de forma permanente; la capa de acceso a datos traduce entre los objetos del programa y las tablas de la base de datos. En BiblioUTEQ la primera es PostgreSQL 18 y la segunda está formada por las entidades JPA y los repositorios de Spring Data.

4.1 El esquema como código: migraciones con Flyway

El esquema de la base de datos (tablas, restricciones e índices) cambia con el proyecto, igual que el código. Si esos cambios se hacen a mano, cada integrante del equipo termina con una base de datos distinta. Una **migración** es un archivo SQL numerado que describe un cambio del esquema; la herramienta Flyway aplica, al arrancar la aplicación, las migraciones que aún no se han ejecutado en esa base de datos y registra cuáles aplicó. En la práctica: nunca se edita una migración ya aplicada; cada cambio nuevo es un archivo nuevo (`V3__...sql`).

Listado 4.1: Migración `V1__esquema_inicial.sql`

```sql
-- V1: esquema inicial de BiblioUTEQ (PostgreSQL 18)

CREATE TABLE socio (
    id        BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    codigo    VARCHAR(20)  NOT NULL UNIQUE,
    nombres   VARCHAR(120) NOT NULL,
    correo    VARCHAR(120) NOT NULL UNIQUE,
    activo    BOOLEAN      NOT NULL DEFAULT TRUE
);

CREATE TABLE libro (
    id                    BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    isbn                  VARCHAR(17)  NOT NULL UNIQUE,
    titulo                VARCHAR(200) NOT NULL,
    autor                 VARCHAR(150) NOT NULL,
    anio                  SMALLINT     NOT NULL,
    ejemplares_totales    INTEGER      NOT NULL CHECK (ejemplares_totales >= 0),
    ejemplares_disponibles INTEGER     NOT NULL,
    version               BIGINT       NOT NULL DEFAULT 0,
    CONSTRAINT ck_libro_disponibles
        CHECK (ejemplares_disponibles BETWEEN 0 AND ejemplares_totales)
);

CREATE TABLE prestamo (
    id               BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    libro_id         BIGINT        NOT NULL REFERENCES libro (id),
    socio_id         BIGINT        NOT NULL REFERENCES socio (id),
    fecha_prestamo   DATE          NOT NULL,
    fecha_vencimiento DATE         NOT NULL,
    fecha_devolucion  DATE,
    estado           VARCHAR(12)   NOT NULL CHECK (estado IN ('ACTIVO', 'DEVUELTO')),
    multa            NUMERIC(8, 2) NOT NULL DEFAULT 0,
    CONSTRAINT ck_prestamo_fechas CHECK (fecha_vencimiento >= fecha_prestamo)
);

-- Indice parcial: solo los prestamos activos, que son los que se consultan para aplicar
```

```
  ↪ las reglas
CREATE INDEX ix_prestamo_socio_activo ON prestamo (socio_id) WHERE estado = 'ACTIVO';
CREATE INDEX ix_prestamo_libro ON prestamo (libro_id);
```

Tres decisiones del listado 4.1 merecen atención:

- Las restricciones `CHECK` hacen que la propia base de datos rechace un estado imposible, como un número negativo de ejemplares disponibles. Es una defensa en profundidad: aunque un error en el código lo intentara, el dato inválido no llegaría a guardarse. El capítulo 9 lo comprueba con una prueba.
- La columna `version` de la tabla `libro` sirve para el control de concurrencia optimista que se explica en la sección 4.5.
- El índice parcial `ix_prestamo_socio_activo` solo indexa los préstamos activos, que son los que se consultan para aplicar la regla del límite de préstamos por socio. Es más pequeño y rápido que un índice sobre toda la tabla.

4.2 Entidades JPA

Una **entidad** es una clase cuyos objetos se guardan como filas de una tabla. JPA (*Jakarta Persistence*) define las anotaciones que indican esa correspondencia, e Hibernate es la implementación que genera el SQL. Este mecanismo es una aplicación del patrón *Data Mapper*: los objetos del dominio no saben cómo se guardan [2].

Listado 4.2: Entidad `Libro` (sin los métodos de acceso)

```
@Entity
@Table(name = "libro")
public class Libro {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 17)
    private String isbn;

    @Column(nullable = false, length = 200)
    private String titulo;

    @Column(nullable = false, length = 150)
    private String autor;

    @Column(nullable = false)
    private short anio;

    @Column(name = "ejemplares_totales", nullable = false)
    private int ejemplaresTotales;

    @Column(name = "ejemplares_disponibles", nullable = false)
    private int ejemplaresDisponibles;

    /** Control de concurrencia optimista: Hibernate incrementa y compara este valor. */
    @Version
    private long version;

    protected Libro() {
        // requerido por JPA
```

```
33      }
34
35      public Libro(String isbn, String titulo, String autor, short anio, int
    ↪ ejemplaresTotales) {
36          if (ejemplaresTotales < 0) {
37              throw new ReglaNegocioException("EJEMPLARES_INVALIDOS", "El numero de
    ↪ ejemplares no puede ser negativo");
38          }
39          this.isbn = isbn;
40          this.titulo = titulo;
41          this.autor = autor;
42          this.anio = anio;
43          this.ejemplaresTotales = ejemplaresTotales;
44          this.ejemplaresDisponibles = ejemplaresTotales;
45      }
46
47      /** Regla de dominio: solo se presta si queda al menos un ejemplar. */
48      public void retirarEjemplar() {
49          if (ejemplaresDisponibles == 0) {
50              throw new ReglaNegocioException("SIN_EJEMPLARES",
51                      "El libro '" + titulo + "' no tiene ejemplares disponibles");
52          }
53          ejemplaresDisponibles--;
54      }
55
56      public void devolverEjemplar() {
57          if (ejemplaresDisponibles == ejemplaresTotales) {
58              throw new ReglaNegocioException("DEVOLUCION_INVALIDA",
59                      "Todos los ejemplares del libro ya estan en la biblioteca");
60          }
61          ejemplaresDisponibles++;
62      }
63
64      public void actualizarDatos(String titulo, String autor, short anio, int
    ↪ ejemplaresTotales) {
65          int prestados = this.ejemplaresTotales - this.ejemplaresDisponibles;
66          if (ejemplaresTotales < prestados) {
67              throw new ReglaNegocioException("EJEMPLARES_INVALIDOS",
68                      "No se puede reducir el total por debajo de los " + prestados + "
    ↪ ejemplares prestados");
69          }
70          this.titulo = titulo;
71          this.autor = autor;
72          this.anio = anio;
73          this.ejemplaresTotales = ejemplaresTotales;
74          this.ejemplaresDisponibles = ejemplaresTotales - prestados;
75      }
```

La entidad del listado 4.2 no es un simple contenedor de datos: los métodos `retirarEjemplar()` y `devolverEjemplar()` protegen la regla de que los ejemplares disponibles nunca son negativos ni mayores que el total. Esta es la idea del modelo de dominio: la regla vive junto a los datos que protege [2], [13]. El constructor sin argumentos es `protected` porque solo lo usa JPA; el resto del código debe usar el constructor que valida los datos.

4.3 Repositorios con Spring Data JPA

Un **repositorio** se comporta como una colección de objetos del dominio en memoria: se le piden, se le agregan y se le quitan objetos, sin escribir SQL [2], [13]. Con Spring Data JPA basta declarar una interfaz; Spring genera la implementación al arrancar, a partir del nombre

de cada método.

Listado 4.3: Repositorio de libros: consultas derivadas del nombre del método

```java
/** Patron Repository: Spring Data genera la implementacion en tiempo de ejecucion. */
public interface LibroRepositorio extends JpaRepository<Libro, Long> {

    Optional<Libro> findByIsbn(String isbn);

    boolean existsByIsbn(String isbn);

    List<Libro> findByTituloContainingIgnoreCaseOrderByTitulo(String texto);

    List<Libro> findAllByOrderByTitulo();
}
```

Por ejemplo, `findByTituloContainingIgnoreCaseOrderByTitulo` genera una consulta que busca el texto dentro del título sin distinguir mayúsculas y ordena por título. Cuando la consulta es más compleja, se escribe en JPQL con `@Query`, como en el listado 4.4.

Listado 4.4: Consultas JPQL con `join fetch` en `PrestamoRepositorio`

```java
public interface PrestamoRepositorio extends JpaRepository<Prestamo, Long> {

    long countBySocioIdAndEstado(Long socioId, EstadoPrestamo estado);

    /** Trae el libro y el socio en la misma consulta para evitar el problema N+1. */
    @Query("""
            select p from Prestamo p
              join fetch p.libro
              join fetch p.socio
             where p.socio.id = :socioId
             order by p.fechaPrestamo desc, p.id desc
            """)
    List<Prestamo> buscarPorSocio(@Param("socioId") Long socioId);

    @Query("""
            select p from Prestamo p
              join fetch p.libro
              join fetch p.socio
             where p.id = :id
            """)
    Optional<Prestamo> buscarConDetalle(@Param("id") Long id);
}
```

El problema de las N+1 consultas

Las relaciones de `Prestamo` con `Libro` y `Socio` se cargan de forma diferida (`FetchType.LAZY`): Hibernate no trae el libro hasta que alguien lo pide. Si se listan 50 préstamos y luego se pide el título de cada libro, se ejecutan 1 + 50 consultas. El `join fetch` del listado 4.4 trae préstamos, libros y socios en una sola consulta. En la práctica, active `spring.jpa.show-sql=true` durante el desarrollo y cuente las consultas que genera cada pantalla.

4.4 Configuración del acceso a datos

La conexión se configura en `application.yaml` con valores que se pueden reemplazar por variables de entorno. Así el mismo artefacto funciona en el equipo del desarrollador (`localhost`)

y en un contenedor (`db`), sin recompilar.

Listado 4.5: Configuración de la fuente de datos y de JPA en `biblio-api`

```yaml
spring:
  application:
    name: biblio-api
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/biblio}
    username: ${DB_USER:biblio}
    password: ${DB_PASSWORD:biblio}
    hikari:
      maximum-pool-size: 10
      connection-timeout: 5000
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate
  flyway:
    enabled: true
  mvc:
    problemdetails:
      enabled: true
```

Tres propiedades son importantes en una aplicación distribuida:

- `hikari.maximum-pool-size` limita las conexiones simultáneas a la base de datos. Abrir una conexión es costoso; el *pool* las reutiliza. Si se replica el API en varios contenedores, la suma de sus *pools* no debe superar el máximo de conexiones de PostgreSQL.
- `ddl-auto: validate` hace que Hibernate compare las entidades con el esquema creado por Flyway y detenga el arranque si no coinciden, en lugar de modificar el esquema por su cuenta.
- `open-in-view: false` cierra la sesión de base de datos al terminar el servicio. Obliga a traer en la capa de negocio todo lo que la respuesta necesita, y evita que una vista dispare consultas inesperadas.

4.5 Concurrencia optimista

En un sistema distribuido, dos usuarios pueden pedir el último ejemplar de un libro en el mismo instante. Si ambas transacciones leen «1 disponible», ambas lo reducen a cero y ambas confirman, se habrá prestado un ejemplar que no existe. El **control de concurrencia optimista** supone que los conflictos son raros y, en lugar de bloquear el dato, comprueba al confirmar si alguien lo modificó mientras tanto [23].

En JPA se activa con un campo anotado con `@Version` (listado 4.2). Hibernate incrementa la versión en cada modificación y la agrega a la condición del `UPDATE`, como en esta sentencia simplificada:

```sql
UPDATE libro SET ejemplares_disponibles = 0, version = 8
 WHERE id = 3 AND version = 7;
```

Si otra transacción ya cambió la versión a 8, la sentencia no actualiza ninguna fila, Hibernate lanza una excepción y la transacción se revierte. El API la traduce en una respuesta HTTP 409 (capítulo 6) y el cliente puede reintentar. La prueba de integración del capítulo 9 lanza tres solicitudes simultáneas por el único ejemplar de un libro y comprueba que solo una tiene

éxito.

Práctica 4.1: una migración nueva

Agregue una migración `V3__categoria_libro.sql` que añada a la tabla `libro` la columna `categoria VARCHAR(60)` y un índice sobre ella. Agregue el atributo a la entidad `Libro`, reinicie `biblio-api` y compruebe en la tabla `flyway_schema_history` que la migración se aplicó. Explique qué ocurriría si, en lugar de crear `V3`, hubiera editado `V1`.

CAPÍTULO 5

Capa de negocio

La capa de negocio responde a la pregunta «¿qué puede hacer el sistema y bajo qué reglas?». En BiblioUTEQ los casos de uso son registrar un préstamo, registrar una devolución y consultar los préstamos de un socio. Esta capa no sabe si la petición llegó por HTTP, por una cola de mensajes o desde una prueba, ni cómo se guardan los datos: solo aplica las reglas y decide qué se confirma.

5.1 Reglas del negocio

Tabla 5.1: Reglas del negocio de BiblioUTEQ

N.º	Regla	Dónde se aplica
R1	Un socio inactivo no puede pedir préstamos.	`Prestamo.registrar` (dominio)
R2	Un socio tiene como máximo 3 préstamos activos.	`PrestamoServicio.registrar` (negocio)
R3	Solo se presta un libro con ejemplares disponibles.	`Libro.retirarEjemplar` (dominio)
R4	El préstamo vence a los 7 días.	`Prestamo.registrar` con el parámetro de configuración
R5	Por cada día de atraso se cobran 0,25 dólares.	`Prestamo.registrarDevolucion`
R6	Un préstamo devuelto no se puede volver a devolver.	`Prestamo.registrarDevolucion`

Las reglas que dependen solo de un objeto (R1, R3, R5, R6) viven en el dominio; la que necesita consultar otros datos (R2, que cuenta los préstamos activos del socio) vive en el servicio. Esta distribución sigue la idea de que el servicio coordina y el dominio decide [2], [13].

5.2 El método fábrica del préstamo

Listado 5.1: Creación y devolución de un préstamo en la entidad `Prestamo`

```
    private Prestamo(Libro libro, Socio socio, LocalDate hoy, int diasPrestamo) {
        this.libro = libro;
        this.socio = socio;
        this.fechaPrestamo = hoy;
        this.fechaVencimiento = hoy.plusDays(diasPrestamo);
        this.estado = EstadoPrestamo.ACTIVO;
    }

    /**
     * Metodo fabrica (patron Factory Method): crea un prestamo valido y retira el
↪ ejemplar.
```

```
     * Es la unica forma de construir un prestamo desde fuera del paquete.
     */
    public static Prestamo registrar(Libro libro, Socio socio, LocalDate hoy, int
  ↪ diasPrestamo) {
        if (!socio.isActivo()) {
            throw new ReglaNegocioException("SOCIO_INACTIVO", "El socio " + socio.
  ↪ getCodigo() + " no esta activo");
        }
        libro.retirarEjemplar();
        return new Prestamo(libro, socio, hoy, diasPrestamo);
    }

    /** Registra la devolucion, calcula la multa por atraso y repone el ejemplar. */
    public void registrarDevolucion(LocalDate hoy, BigDecimal multaPorDia) {
        if (estado == EstadoPrestamo.DEVUELTO) {
            throw new ReglaNegocioException("YA_DEVUELTO", "El prestamo " + id + " ya fue
  ↪  devuelto");
        }
        long diasAtraso = Math.max(0, ChronoUnit.DAYS.between(fechaVencimiento, hoy));
        this.multa = multaPorDia.multiply(BigDecimal.valueOf(diasAtraso)).setScale(2,
  ↪ RoundingMode.HALF_UP);
        this.fechaDevolucion = hoy;
        this.estado = EstadoPrestamo.DEVUELTO;
        libro.devolverEjemplar();
    }
```

El constructor de `Prestamo` es privado: la única forma de crear un préstamo desde otra clase es el método fábrica `registrar` [12], que valida al socio y retira el ejemplar antes de construir el objeto. Así es imposible, por diseño, que exista un préstamo sin su ejemplar retirado.

5.3 El servicio de préstamos

Listado 5.2: Servicio `PrestamoServicio`: casos de uso y transacciones

```
/**
 * Capa de negocio: orquesta el caso de uso "prestar un libro".
 * Cada metodo publico es una transaccion: o se aplican todos los cambios, o ninguno.
 */
@Service
@Transactional(readOnly = true)
public class PrestamoServicio {

    private final PrestamoRepositorio prestamos;
    private final LibroRepositorio libros;
    private final SocioRepositorio socios;
    private final ReglasPrestamo reglas;
    private final Clock reloj;
    private final ApplicationEventPublisher eventos;

    public PrestamoServicio(PrestamoRepositorio prestamos, LibroRepositorio libros,
  ↪ SocioRepositorio socios,
            ReglasPrestamo reglas, Clock reloj, ApplicationEventPublisher eventos) {
        this.prestamos = prestamos;
        this.libros = libros;
        this.socios = socios;
        this.reglas = reglas;
        this.reloj = reloj;
        this.eventos = eventos;
    }

```

```
26      @Transactional
27      public Prestamo registrar(Long socioId, Long libroId) {
28          Socio socio = socios.findById(socioId)
29                  .orElseThrow(() -> new RecursoNoEncontradoException("Socio", socioId));
30
31          long activos = prestamos.countBySocioIdAndEstado(socioId, EstadoPrestamo.ACTIVO);
32          if (activos >= reglas.maxPrestamosActivos()) {
33              throw new ReglaNegocioException("LIMITE_PRESTAMOS",
34                      "El socio ya tiene " + activos + " prestamos activos (maximo " +
    ↪ reglas.maxPrestamosActivos() + ")");
35          }
36
37          Libro libro = libros.findById(libroId)
38                  .orElseThrow(() -> new RecursoNoEncontradoException("Libro", libroId));
39
40          Prestamo prestamo = Prestamo.registrar(libro, socio, LocalDate.now(reloj), reglas.
    ↪ diasPrestamo());
41          Prestamo guardado = prestamos.save(prestamo);
42
43          eventos.publishEvent(new PrestamoRegistradoEvento(guardado.getId(), socio.
    ↪ getCodigo(),
44                  libro.getIsbn(), guardado.getFechaVencimiento()));
45          return guardado;
46      }
47
48      @Transactional
49      public Prestamo devolver(Long prestamoId) {
50          Prestamo prestamo = prestamos.buscarConDetalle(prestamoId)
51                  .orElseThrow(() -> new RecursoNoEncontradoException("Prestamo",
    ↪ prestamoId));
52          prestamo.registrarDevolucion(LocalDate.now(reloj), reglas.multaPorDia());
53          return prestamo;
54      }
55
56      public List<Prestamo> listarPorSocio(Long socioId) {
57          if (!socios.existsById(socioId)) {
58              throw new RecursoNoEncontradoException("Socio", socioId);
59          }
60          return prestamos.buscarPorSocio(socioId);
61      }
62  }
```

5.3.1 Transacciones

Una **transacción** agrupa varias operaciones sobre la base de datos para que se apliquen todas o ninguna. En `registrar`, retirar el ejemplar y guardar el préstamo forman una unidad: si guardar el préstamo falla, el ejemplar no debe quedar retirado. La anotación `@Transactional` delimita esa unidad: Spring abre la transacción al entrar al método, la confirma al salir y la revierte si se lanza una excepción no comprobada, como `ReglaNegocioException`.

La clase entera tiene la anotación `@Transactional` con el atributo `readOnly` activado, y los métodos que modifican datos la sobrescriben con `@Transactional`. La marca de solo lectura permite al proveedor JPA omitir la detección de cambios en las consultas. Observe además que `devolver` no llama a ningún método `save`: la entidad cargada está administrada por JPA y sus cambios se escriben al confirmar la transacción.

5.3.2 Reglas configurables y reloj inyectado

Los valores de las reglas (máximo de préstamos, días de préstamo, multa diaria) no están escritos en el código: se leen de la configuración mediante un registro de Java con la anotación `@ConfigurationProperties`, y se validan al arrancar la aplicación.

Listado 5.3: Parámetros del negocio leídos de la configuración

```
/**
 * Parametros del negocio leidos de application.yaml (prefijo biblio.reglas).
 * Cambiar una regla no exige recompilar: basta con cambiar la configuracion.
 */
@Validated
@ConfigurationProperties(prefix = "biblio.reglas")
public record ReglasPrestamo(
        @Min(1) int maxPrestamosActivos,
        @Min(1) int diasPrestamo,
        @DecimalMin("0.00") BigDecimal multaPorDia) {
}
```

El servicio tampoco pide la fecha con `LocalDate.now()`, sino a un objeto `Clock` que recibe por inyección. En producción ese reloj es el del sistema; en una prueba es un reloj fijo, de modo que el cálculo de la multa da siempre el mismo resultado, sin importar el día en que se ejecute la prueba.

5.4 Eventos del dominio: el patrón Observador

Cuando se confirma un préstamo, otras partes del sistema podrían querer enterarse: enviar un correo al socio, actualizar estadísticas o publicar el hecho en un *broker* de mensajes. Si el servicio llamara a cada una, crecería con cada nueva necesidad. Con el patrón Observador, el servicio publica un evento y los interesados se suscriben a él sin que el servicio los conozca [12].

Listado 5.4: Observador que reacciona a los préstamos confirmados

```
/**
 * Observador (patron Observer): reacciona a los prestamos confirmados sin que
 * PrestamoServicio lo conozca. Solo se ejecuta si la transaccion se confirmo.
 * En un sistema real enviaria un correo o publicaria el evento en un broker.
 */
@Component
public class NotificadorPrestamos {

    private static final Logger LOG = LoggerFactory.getLogger(NotificadorPrestamos.class);

    @TransactionalEventListener
    public void alRegistrarPrestamo(PrestamoRegistradoEvento evento) {
        LOG.info("Prestamo {} confirmado: socio={}, isbn={}, vence={}",
                evento.prestamoId(), evento.codigoSocio(), evento.isbn(), evento.
    fechaVencimiento());
    }
}
```

La anotación `@TransactionalEventListener` hace que el observador se ejecute solo después de que la transacción se confirma. Si el préstamo se revierte, no se notifica nada, lo que evita avisar al socio de un préstamo que no existe.

Una condición de carrera que queda abierta

La regla R2 cuenta los préstamos activos y luego inserta uno nuevo. Si el mismo socio envía dos solicitudes al mismo tiempo para dos libros distintos, ambas transacciones pueden contar 2, ambas pasar la validación y el socio terminar con 4 préstamos. El control optimista sobre `Libro` no lo impide, porque cada solicitud modifica un libro diferente. La práctica 5.1 pide corregirlo.

Práctica 5.1: cerrar la condición de carrera de la regla R2

Proponga y aplique una de estas soluciones y justifique su elección: (a) bloquear la fila del socio al inicio de `registrar` con `@Lock(LockModeType.PESSIMISTIC_WRITE)` en una consulta del repositorio de socios; (b) agregar a `Socio` un contador de préstamos activos protegido con `@Version`; (c) ejecutar la transacción con aislamiento serializable. Escriba una prueba de integración que lance dos solicitudes simultáneas del mismo socio con dos préstamos activos y compruebe que solo una tiene éxito. Discuta qué cuesta cada solución en latencia y en reintentos [5].

CAPÍTULO 6

Capa de servicios: el API REST

La capa de servicios publica los casos de uso del negocio para que otros niveles los usen a través de la red. En BiblioUTEQ es un API REST: un conjunto de recursos identificados por URL que se manipulan con los métodos de HTTP e intercambian representaciones en JSON. Esta capa es delgada a propósito: recibe, valida, delega en la capa de negocio y traduce el resultado a HTTP.

6.1 Diseño de los recursos

El estilo REST, definido por Fielding, organiza un sistema en recursos con identificadores uniformes, sobre los que se aplica un conjunto reducido de operaciones con semántica conocida, sin que el servidor guarde el estado de la conversación con cada cliente [24]. La semántica de cada método y de cada código de estado la fija la especificación de HTTP [25]. La tabla 6.1 muestra los recursos de BiblioUTEQ.

Tabla 6.1: Recursos del API de BiblioUTEQ

Método	Ruta	Operación	Éxito
GET	`/api/v1/libros?q=texto`	Listar o buscar libros	200
GET	`/api/v1/libros/{id}`	Obtener un libro	200
POST	`/api/v1/libros`	Registrar un libro	201
PUT	`/api/v1/libros/{id}`	Reemplazar los datos de un libro	200
GET	`/api/v1/socios`	Listar socios	200
GET	`/api/v1/socios/{id}/prestamos`	Préstamos de un socio	200
POST	`/api/v1/prestamos`	Registrar un préstamo	201
POST	`/api/v1/prestamos/{id}/devolucion`	Registrar la devolución	200

Tres decisiones de diseño explican la tabla:

- **Sustantivos, no verbos.** Las rutas nombran recursos (`/prestamos`); la acción la indica el método HTTP. Seguir reglas de diseño como esta mejora la comprensión de un API por parte de quienes la usan [26].
- **La devolución es un subrecurso.** Devolver un libro no es simplemente cambiar un campo: calcula la multa y repone el ejemplar. Por eso se modela como una acción explícita con `POST`, en lugar de permitir que el cliente envíe el estado «DEVUELTO» con `PUT`.
- **Versión en la ruta.** El prefijo `/v1` permite publicar más adelante una `/v2` incompatible sin romper a los clientes existentes. Cómo evolucionar un API sin romper a sus consumidores es uno de los retos más frecuentes en sistemas distribuidos en la práctica [27].

6.2 Objetos de transferencia de datos y validación

El API nunca devuelve las entidades JPA. Usa **objetos de transferencia de datos** (DTO) [2], escritos como `record` de Java: inmutables, sin lógica y con exactamente los campos del contrato. Así, agregar una columna interna a la tabla no cambia el contrato del API, y no se exponen por accidente datos internos como el campo `version`.

Listado 6.1: DTO de entrada con reglas de validación

```
/** Datos que el cliente envia para crear o actualizar un libro. */
public record LibroSolicitud(
        @NotBlank @Pattern(regexp = "[0-9-]{10,17}", message = "debe contener solo
  ↪ digitos y guiones") String isbn,
        @NotBlank @Size(max = 200) String titulo,
        @NotBlank @Size(max = 150) String autor,
        @Min(1450) @Max(2100) short anio,
        @Min(0) @Max(1000) int ejemplaresTotales) {
}
```

Las anotaciones de Jakarta Validation del listado 6.1 se comprueban antes de llamar al servicio cuando el parámetro del controlador lleva `@Valid`. Un dato inválido nunca llega a la capa de negocio.

6.3 Los controladores

Listado 6.2: Controlador de préstamos

```
@RestController
@RequestMapping("/api/v1/prestamos")
public class PrestamoControlador {

    private final PrestamoServicio servicio;

    public PrestamoControlador(PrestamoServicio servicio) {
        this.servicio = servicio;
    }

    @PostMapping
    public ResponseEntity<PrestamoRespuesta> registrar(@Valid @RequestBody
  ↪ PrestamoSolicitud s) {
        Prestamo prestamo = servicio.registrar(s.socioId(), s.libroId());
        URI ubicacion = URI.create("/api/v1/prestamos/" + prestamo.getId());
        return ResponseEntity.created(ubicacion).body(PrestamoRespuesta.desde(prestamo));
    }

    /** La devolucion es una accion sobre el recurso: se modela como subrecurso con POST.
  ↪  */
    @PostMapping("/{id}/devolucion")
    public PrestamoRespuesta devolver(@PathVariable Long id) {
        return PrestamoRespuesta.desde(servicio.devolver(id));
    }
}
```

El controlador del listado 6.2 hace solo tres cosas: recibe el DTO validado, llama al servicio y convierte el resultado en un DTO de respuesta con el código adecuado. Al crear un recurso responde `201 Created` con la cabecera `Location`, que indica la URL del recurso nuevo [25].

6.4 Errores con Problem Details (RFC 9457)

Un API distribuido debe informar sus errores de forma que el cliente los pueda procesar, no solo mostrar. El estándar *Problem Details for HTTP APIs* (RFC 9457) define un formato JSON común para los errores, con los campos `type`, `title`, `status`, `detail` e `instance`, que pueden ampliarse con campos propios [28]. Spring lo implementa con la clase `ProblemDetail`.

Listado 6.3: Traducción de excepciones a respuestas HTTP en `ManejadorErrores`

```
/**
 * Traduce las excepciones de las capas inferiores a respuestas HTTP con el formato
 * estandar Problem Details (RFC 9457). Las capas de negocio y datos no conocen HTTP.
 * La clase base ya responde en ese formato a los errores propios de Spring MVC.
 */
@RestControllerAdvice
public class ManejadorErrores extends ResponseEntityExceptionHandler {

    private static final String BASE = "https://biblio.uteq.edu.ec/errores/";

    @ExceptionHandler(RecursoNoEncontradoException.class)
    ProblemDetail noEncontrado(RecursoNoEncontradoException ex) {
        ProblemDetail p = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.
  ↪ getMessage());
        p.setType(URI.create(BASE + "no-encontrado"));
        p.setTitle("Recurso no encontrado");
        return p;
    }

    @ExceptionHandler(ReglaNegocioException.class)
    ProblemDetail reglaNegocio(ReglaNegocioException ex) {
        ProblemDetail p = ProblemDetail.forStatusAndDetail(HttpStatus.
  ↪ UNPROCESSABLE_CONTENT, ex.getMessage());
        p.setType(URI.create(BASE + "regla-negocio"));
        p.setTitle("Regla de negocio incumplida");
        p.setProperty("codigo", ex.getCodigo());
        return p;
    }

    /** Errores de validacion (400): se agrega el detalle de cada campo invalido. */
    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
  ↪ MethodArgumentNotValidException ex,
            HttpHeaders headers, HttpStatusCode status, WebRequest request) {
        ProblemDetail p = ex.getBody();
        p.setTitle("Datos invalidos");
        p.setDetail("La solicitud tiene campos con valores no validos");
        Map<String, String> errores = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(e -> errores.put(e.getField(), e.
  ↪ getDefaultMessage()));
        p.setProperty("errores", errores);
        return handleExceptionInternal(ex, p, headers, status, request);
    }

    /** Dos transacciones modificaron el mismo libro a la vez y esta perdio: el cliente
  ↪ puede reintentar. */
    @ExceptionHandler(ObjectOptimisticLockingFailureException.class)
    ProblemDetail conflicto(ObjectOptimisticLockingFailureException ex) {
        ProblemDetail p = ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT,
                "El recurso fue modificado por otra operacion al mismo tiempo; intente de
  ↪  nuevo");
        p.setType(URI.create(BASE + "conflicto-concurrencia"));
        p.setTitle("Conflicto de concurrencia");
```

```
48          return p;
49      }
50  }
```

La tabla 6.2 resume la correspondencia entre las excepciones de las capas inferiores y los códigos HTTP. Fíjese en que las capas de negocio y de dominio lanzan sus propias excepciones sin saber nada de HTTP: la traducción ocurre solo en esta capa.

Tabla 6.2: Correspondencia entre errores y códigos HTTP

Situación	Código	Significado para el cliente
Datos de entrada inválidos	400	Corregir la solicitud; el campo `errores` indica qué falló
`RecursoNoEncontradoException`	404	El recurso pedido no existe
`ObjectOptimisticLockingFailureException`	409	Otro usuario modificó el dato a la vez; reintentar
`ReglaNegocioException`	422	La solicitud es válida, pero una regla del negocio la impide; el campo `codigo` dice cuál

El código 422 se llama *Unprocessable Content* en la especificación vigente de HTTP [25]; Spring Framework 7 lo nombra `HttpStatus.UNPROCESSABLE_CONTENT`. Estas son respuestas reales del API en ejecución:

Listado 6.4: Respuestas reales del API ante tres situaciones

```
POST /api/v1/prestamos  {"socioId":1,"libroId":3}
HTTP/1.1 201
Location: /api/v1/prestamos/1
{"id":1,"socioId":1,"codigoSocio":"E-0001","libroId":3,"tituloLibro":"Building
  ↪ Microservices","fechaPrestamo":"2026-10-26","fechaVencimiento":"2026-11-02","
  ↪ fechaDevolucion":null,"estado":"ACTIVO","multa":0}

POST /api/v1/prestamos  {"socioId":2,"libroId":3}
HTTP/1.1 422
Content-Type: application/problem+json
{"detail":"El libro 'Building Microservices' no tiene ejemplares disponibles","instance
  ↪ ":"/api/v1/prestamos","status":422,"title":"Regla de negocio incumplida","type":"
  ↪ https://biblio.uteq.edu.ec/errores/regla-negocio","codigo":"SIN_EJEMPLARES"}

POST /api/v1/prestamos  {"socioId":-1}
HTTP/1.1 400
Content-Type: application/problem+json
{"detail":"La solicitud tiene campos con valores no validos","instance":"/api/v1/
  ↪ prestamos","status":400,"title":"Datos invalidos","errores":{"libroId":"no debe ser
  ↪ nulo","socioId":"debe ser mayor que 0"}}
```

6.5 El contrato OpenAPI

El contrato de un API es la descripción precisa de sus rutas, parámetros, esquemas y códigos de respuesta. La especificación OpenAPI es el estándar más usado para escribir ese contrato en un formato que leen tanto las personas como las herramientas; su versión más reciente es la

3.2.1 [29]. En BiblioUTEQ, springdoc-openapi genera el contrato a partir de los controladores y lo publica en `http://localhost:8080/v3/api-docs`, en formato OpenAPI 3.1, junto con la interfaz interactiva Swagger UI (figura 6.1).

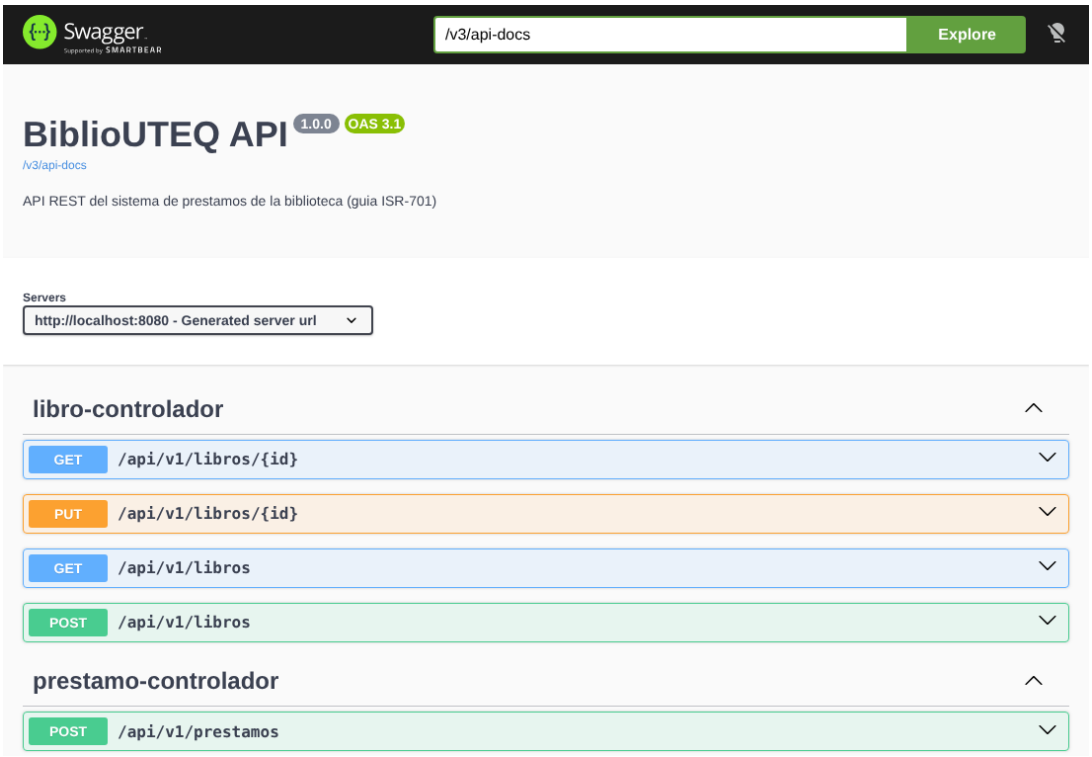


Figura 6.1: Contrato del API publicado con Swagger UI en `biblio-api`

Desde Swagger UI se puede probar cada operación con el botón *Try it out*. Desde PowerShell, el comando `Invoke-RestMethod` permite lo mismo:

```
Invoke-RestMethod http://localhost:8080/api/v1/libros
Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/v1/prestamos `
    -ContentType 'application/json' -Body '{"socioId": 1, "libroId": 3}'
```

Práctica 6.1: un recurso nuevo

Agregue la operación `GET /api/v1/prestamos/vencidos`, que devuelva los préstamos activos cuya fecha de vencimiento ya pasó. Implemente la consulta en el repositorio, el método en el servicio (usando el reloj inyectado) y el método en el controlador. Compruebe que la operación aparece en Swagger UI con su esquema de respuesta y documente en una frase por qué no recibe la fecha actual como parámetro.

CAPÍTULO 7

Capa de presentación distribuida

La capa de presentación de BiblioUTEQ es una aplicación web independiente, `biblio-web`, que se ejecuta en su propio nivel y no tiene acceso a la base de datos: todo lo que muestra lo obtiene del API. Esta separación es la que permitiría reemplazarla por una aplicación móvil o una aplicación de página única sin tocar el negocio. A cambio, cada pantalla depende de un servicio remoto que puede tardar o fallar, y el código debe estar preparado para eso.

7.1 Un cliente HTTP declarativo

Para consumir el API, `biblio-web` declara una interfaz Java con una anotación por operación remota. Spring genera en tiempo de ejecución un objeto que implementa la interfaz y convierte cada llamada en una solicitud HTTP: es un **proxy remoto**, un objeto local que representa a otro que vive en otro proceso [12]. El controlador lo usa como si llamara a un objeto cualquiera.

Listado 7.1: Cliente declarativo del API en `biblio-web`

```
/**
 * Cliente HTTP declarativo. Spring crea en tiempo de ejecucion un objeto que
 * implementa esta interfaz y traduce cada llamada a una solicitud HTTP al API
 * (patron Proxy remoto): el controlador lo usa como si fuera un objeto local.
 */
@HttpExchange("/api/v1")
public interface BiblioApiCliente {

    @GetExchange("/libros")
    List<LibroDatos> listarLibros(@RequestParam(name = "q", required = false) String
  ↪ texto);

    @GetExchange("/socios")
    List<SocioDatos> listarSocios();

    @GetExchange("/socios/{id}/prestamos")
    List<PrestamoDatos> prestamosDelSocio(@PathVariable Long id);

    @PostExchange("/prestamos")
    PrestamoDatos registrarPrestamo(@RequestBody SolicitudPrestamo solicitud);

    @PostExchange("/prestamos/{id}/devolucion")
    PrestamoDatos devolver(@PathVariable Long id);
}
```

Observe que `biblio-web` define sus propios registros (`LibroDatos`, `PrestamoDatos`) en lugar de reutilizar las clases de `biblio-api`. Los dos niveles comparten solo el contrato JSON, no el código: así cada uno puede compilarse, versionarse y desplegarse por separado.

7.2 Tiempos de espera: la primera defensa ante fallos parciales

Dentro de un solo proceso, una llamada a un método siempre termina. Entre niveles no: si el API está caído o saturado, una solicitud HTTP sin límite de tiempo puede dejar esperando al usuario indefinidamente y agotar los hilos del servidor web. En los sistemas distribuidos estas

fallas parciales son la norma, no la excepción [4]. Por eso el cliente se configura con un tiempo máximo de conexión y otro de lectura.

Listado 7.2: Configuración del cliente con tiempos de espera

```java
/**
 * Configura la comunicacion entre niveles: URL del API (por variable de entorno)
 * y tiempos de espera. Sin tiempos de espera, una caida del API dejaria colgadas
 * las solicitudes de los usuarios de la aplicacion web.
 */
@Configuration
public class ClienteApiConfig {

    @Bean
    BiblioApiCliente biblioApiCliente(RestClient.Builder builder,
            @Value("${biblio.api.url}") String urlApi,
            @Value("${biblio.api.timeout-conexion:2s}") Duration timeoutConexion,
            @Value("${biblio.api.timeout-lectura:5s}") Duration timeoutLectura) {

        HttpClient http = HttpClient.newBuilder().connectTimeout(timeoutConexion).build();
        JdkClientHttpRequestFactory fabrica = new JdkClientHttpRequestFactory(http);
        fabrica.setReadTimeout(timeoutLectura);

        RestClient restClient = builder.baseUrl(urlApi).requestFactory(fabrica).build();
        return HttpServiceProxyFactory.builderFor(RestClientAdapter.create(restClient))
                .build()
                .createClient(BiblioApiCliente.class);
    }
}
```

La URL del API se lee de la propiedad `biblio.api.url`, que en el contenedor se define con la variable de entorno `BIBLIO_API_URL`. El mismo artefacto funciona en el equipo del desarrollador y en Docker Compose.

7.3 Controladores web y el patrón Post/Redirect/Get

Listado 7.3: Controlador web de préstamos

```java
@Controller
public class PrestamoWebControlador {

    private final BiblioApiCliente api;

    public PrestamoWebControlador(BiblioApiCliente api) {
        this.api = api;
    }

    @GetMapping("/prestamos")
    public String prestamos(@RequestParam(name = "socioId", required = false) Long socioId, Model modelo) {
        modelo.addAttribute("socios", api.listarSocios());
        modelo.addAttribute("libros", api.listarLibros(null));
        modelo.addAttribute("socioId", socioId);
        List<PrestamoDatos> lista = socioId == null ? List.of() : api.prestamosDelSocio(socioId);
        modelo.addAttribute("prestamos", lista);
        return "prestamos";
    }

```

```
    @PostMapping("/prestamos")
    public String registrar(@RequestParam Long socioId, @RequestParam Long libroId,
  ↪ RedirectAttributes flash) {
        try {
            PrestamoDatos p = api.registrarPrestamo(new SolicitudPrestamo(socioId,
  ↪ libroId));
            flash.addFlashAttribute("exito", "Pr\u00e9stamo registrado. Vence el " + p.
  ↪ fechaVencimiento() + ".");
        } catch (HttpClientErrorException ex) {
            flash.addFlashAttribute("error", detalle(ex));
        }
        return "redirect:/prestamos?socioId=" + socioId;
    }

    @PostMapping("/prestamos/{id}/devolucion")
    public String devolver(@PathVariable Long id, @RequestParam Long socioId,
  ↪ RedirectAttributes flash) {
        try {
            PrestamoDatos p = api.devolver(id);
            flash.addFlashAttribute("exito", "Devoluci\u00f3n registrada. Multa: $" + p.
  ↪ multa() + ".");
        } catch (HttpClientErrorException ex) {
            flash.addFlashAttribute("error", detalle(ex));
        }
        return "redirect:/prestamos?socioId=" + socioId;
    }

    /** Muestra al usuario el campo "detail" del Problem Details que devolvio el API. */
    private static String detalle(HttpClientErrorException ex) {
        ProblemDetail problema = ex.getResponseBodyAs(ProblemDetail.class);
        return problema != null && problema.getDetail() != null ? problema.getDetail() :
  ↪ ex.getStatusText();
    }
}
```

Después de procesar un formulario, el controlador no devuelve una página: redirige a una URL de consulta. Este patrón, conocido como Post/Redirect/Get, evita que el préstamo se registre dos veces si el usuario recarga la página. El mensaje de éxito o de error viaja a la página siguiente como atributo *flash*, que se borra después de mostrarse.

Cuando el API rechaza la operación por una regla del negocio, el controlador lee el campo `detail` del Problem Details y lo muestra al usuario (figura 7.1). La capa de presentación no repite la regla: solo comunica la decisión que tomó el negocio.

7.4 Cuando el API no responde

Listado 7.4: Manejo de la indisponibilidad del API

```
/**
 * Fallo parcial: si el nivel del API no responde (caido, lento o con error interno),
 * la capa de presentacion sigue viva y muestra una pagina controlada.
 */
@ControllerAdvice
public class ManejadorFallosApi {

    private static final Logger LOG = LoggerFactory.getLogger(ManejadorFallosApi.class);

    @ExceptionHandler({ ResourceAccessException.class, HttpServerErrorException.class })
    @ResponseStatus(HttpStatus.SERVICE_UNAVAILABLE)
```

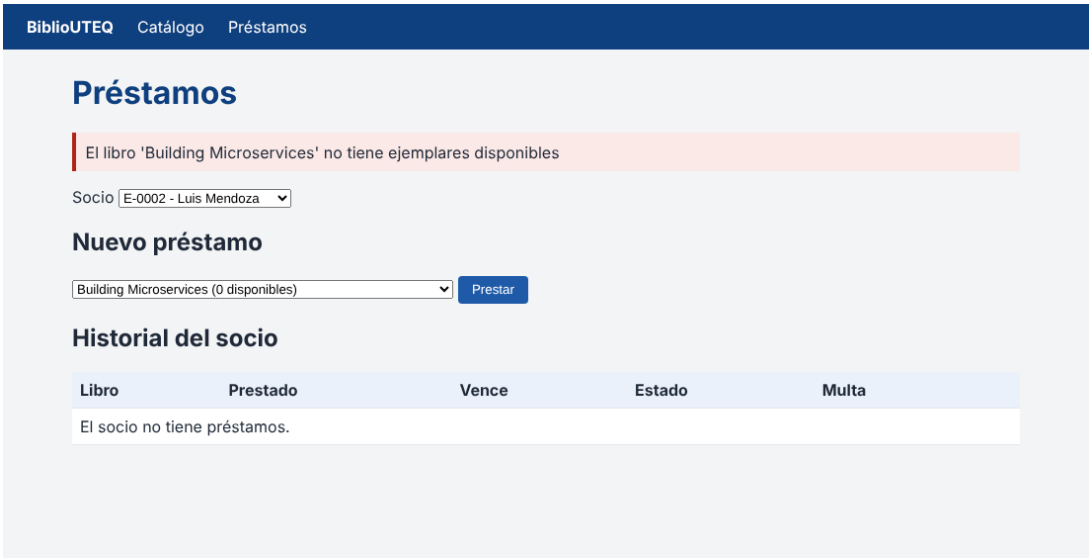


Figura 7.1: La regla «sin ejemplares» decidida por el API y mostrada por la capa de presentación

```
    public String apiNoDisponible(Exception ex) {
        LOG.warn("El API de biblioteca no esta disponible: {}", ex.getMessage());
        return "servicio-no-disponible";
    }
}
```

Si el API está caído, el cliente lanza `ResourceAccessException` al vencer el tiempo de espera, y si responde con un error interno, `HttpServerErrorException`. El manejador del listado 7.4 las convierte en una página controlada con el código 503, en lugar de una traza de error (figura 7.2). La aplicación web sigue viva y se recupera sola cuando el API vuelve.

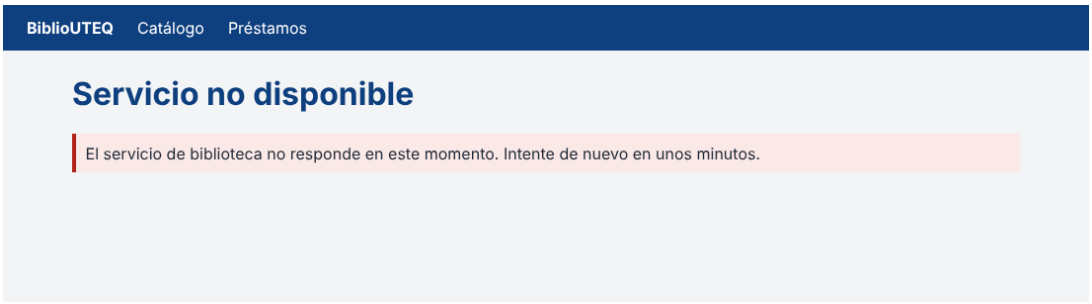


Figura 7.2: Respuesta de `biblio-web` con el contenedor del API detenido

7.5 Las vistas

Las vistas se escriben con Thymeleaf, un motor de plantillas que genera HTML en el servidor. Los atributos `th:*` se reemplazan con los datos del modelo y el resto del archivo es HTML válido que puede abrirse en un navegador como prototipo.

Listado 7.5: Plantilla `catalogo.html`

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="es">
<head th:replace="~{fragmentos :: cabecera('Catalogo')}"></head>
<body>
```

```
<nav th:replace="~{fragmentos :: menu}"></nav>
<main>
    <h1>Cat&aacute;logo de libros</h1>
    <form th:action="@{/}" method="get" class="busqueda">
        <input type="search" name="q" th:value="${q}" placeholder="Buscar por t&iacute;tulo">
        <button type="submit">Buscar</button>
    </form>
    <table>
        <thead>
        <tr><th>ISBN</th><th>T&iacute;tulo</th><th>Autor</th><th>A&ntilde;o</th><th>Disponibles</th></tr>
        </thead>
        <tbody>
        <tr th:each="l : ${libros}">
            <td th:text="${l.isbn}">978-...</td>
            <td th:text="${l.titulo}">Titulo</td>
            <td th:text="${l.autor}">Autor</td>
            <td th:text="${l.anio}">2026</td>
            <td th:text="|${l.ejemplaresDisponibles} de ${l.ejemplaresTotales}|"
                th:classappend="${l.ejemplaresDisponibles == 0} ? 'agotado'">1 de 2</td>
        </tr>
        <tr th:if="${#lists.isEmpty(libros)}"><td colspan="5">No hay libros que coincidan.</td></tr>
        </tbody>
    </table>
</main>
</body>
</html>
```



Figura 7.3: Catálogo de libros en `biblio-web`

Práctica 7.1: reintentos y disyuntor

Los tiempos de espera evitan que el usuario espere para siempre, pero si el API está caído, cada solicitud seguirá intentando conectarse. Agregue a `biblio-web` un disyuntor (*circuit breaker*) con la biblioteca Resilience4j sobre las llamadas de lectura del catálogo: después de varios fallos seguidos, el disyuntor deja de llamar al API durante un tiempo y responde de inmediato con la página de servicio no disponible. Mida, con el API detenido, cuánto tarda la página antes y después del cambio.

CAPÍTULO 8

Integración y despliegue con contenedores

Hasta aquí cada nivel se ejecutó desde IntelliJ IDEA. En este capítulo los tres niveles se empaquetan en contenedores y se levantan juntos con un solo comando. Un contenedor incluye el programa y todo lo que necesita para ejecutarse (el JRE, las bibliotecas y la configuración por defecto), de modo que se comporta igual en el equipo de cualquier integrante del curso, en el servidor de integración continua y en la nube.

8.1 Imágenes de los niveles de aplicación

Cada módulo tiene su `Dockerfile`. La imagen parte de Eclipse Temurin 27 en su versión JRE (solo lo necesario para ejecutar, no para compilar), copia el archivo `.jar` generado por Maven y lo ejecuta con un usuario sin privilegios de administrador.

Listado 8.1: `Dockerfile` de `biblio-api`

```
# Imagen de ejecucion de biblio-api (el .jar se construye antes con: mvnw package)
FROM eclipse-temurin:27-jre

RUN useradd --system --create-home --uid 10001 app
WORKDIR /app
COPY target/biblio-api-1.0.0.jar app.jar

USER app
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

¿Por qué un usuario sin privilegios?

Si un atacante lograra ejecutar código dentro del contenedor, lo haría con los permisos del usuario del proceso. La instrucción `USER app` limita ese daño. Es una de las prácticas de seguridad más simples y efectivas al construir imágenes.

8.2 Orquestación local con Docker Compose

El archivo `compose.yaml` describe los tres niveles, cómo se conectan y en qué orden arrancan.

Listado 8.2: `compose.yaml`: los tres niveles de BiblioUTEQ

```
1  # BiblioUTEQ: tres niveles en tres contenedores
2  #   web (presentacion, 8081) -> api (negocio y datos, 8080) -> db (PostgreSQL 18, 5432)
3
4  services:
5    db:
6      image: postgres:18
7      environment:
8        POSTGRES_DB: biblio
9        POSTGRES_USER: ${DB_USER:-biblio}
10       POSTGRES_PASSWORD: ${DB_PASSWORD:-biblio}
11     ports:
12       - "5432:5432"
```

```
    volumes:
      - datos-biblio:/var/lib/postgresql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-biblio} -d biblio"]
      interval: 5s
      timeout: 3s
      retries: 10

  api:
    build: ./biblio-api
    environment:
      DB_URL: jdbc:postgresql://db:5432/biblio
      DB_USER: ${DB_USER:-biblio}
      DB_PASSWORD: ${DB_PASSWORD:-biblio}
    ports:
      - "8080:8080"
    depends_on:
      db:
        condition: service_healthy

  web:
    build: ./biblio-web
    environment:
      BIBLIO_API_URL: http://api:8080
    ports:
      - "8081:8081"
    depends_on:
      - api

volumes:
  datos-biblio:
```

Puntos clave del listado 8.2:

- **Nombres de servicio como direcciones.** Dentro de la red que crea Compose, el API alcanza a la base de datos en `db:5432` y la aplicación web alcanza al API en `http://api:8080`. Esas direcciones se pasan por variables de entorno, sin cambiar el código.
- **Arranque condicionado a la salud.** El API espera a que PostgreSQL responda a `pg_isready`; si arrancara antes, Flyway fallaría al no poder conectarse.
- **Volumen persistente.** Los datos sobreviven a `docker compose down`. En la imagen oficial de PostgreSQL 18 los datos se guardan en un subdirectorio por versión (`/var/lib/postgresql/18/docker`), por lo que el volumen se monta en `/var/lib/postgresql`.
- **Secretos fuera del código.** Usuario y clave se toman de variables de entorno con valores por defecto para desarrollo. En un despliegue real se definen en un archivo `.env` que no se sube al repositorio.

8.3 Levantar el sistema completo

Desde la carpeta del proyecto, en PowerShell:

```
.\mvnw.cmd -DskipTests package     # genera los dos .jar
docker compose up -d --build       # construye las imagenes y levanta los tres niveles
docker compose ps                  # estado de los contenedores
docker compose logs -f api         # registros del API (Ctrl+C para salir)
```

Al terminar, `http://localhost:8081` muestra la aplicación web, `http://localhost:8080/swagger-ui.html` el contrato del API y `http://localhost:8080/actuator/health` el estado del API, que debe responder `"status":ÜP"`.

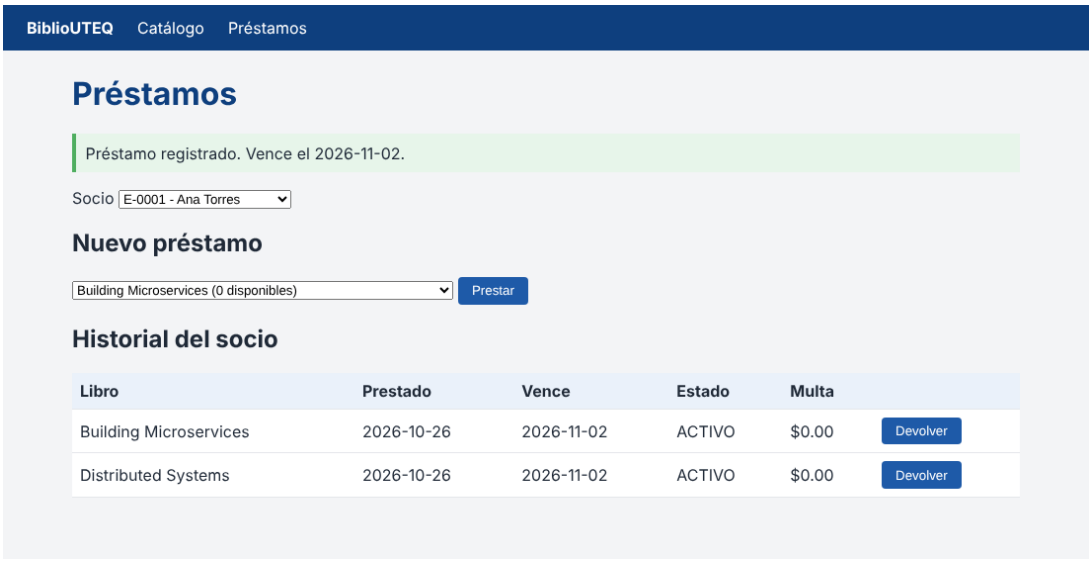


Figura 8.1: Registro de préstamos con los tres niveles ejecutándose en contenedores

8.4 Provocar una falla parcial

Un sistema distribuido se entiende mejor cuando se le hace fallar a propósito. Con el sistema en marcha, detenga solo el API y observe la aplicación web:

```
docker compose stop api      # el nivel de servicios deja de responder
# abrir http://localhost:8081: se muestra la pagina de servicio no disponible
docker compose start api     # el nivel vuelve; la aplicacion web se recupera sola
```

La aplicación web no se cae: espera como máximo el tiempo configurado, responde con la página controlada de la figura 7.2 y vuelve a funcionar en cuanto el API está disponible, sin reiniciarla.

Problemas frecuentes en Windows

- **`port is already allocated`**: otro programa usa el puerto 5432, 8080 u 8081. Si tiene PostgreSQL instalado de forma nativa, deténgalo desde *Servicios* de Windows o cambie el puerto publicado en `compose.yaml` (por ejemplo, `"5433:5432"`).
- **`Cannot connect to the Docker daemon`**: Docker Desktop no está abierto o su motor no terminó de arrancar.
- **`Unable to access jarfile`** al construir la imagen: no se ejecutó `mvnw.cmd package` antes de `docker compose up -build`.
- **Datos de una versión anterior**: si cambió migraciones ya aplicadas, borre el volumen con `docker compose down -v`; se perderán los datos de desarrollo.

Práctica 8.1: escalar el nivel del API

Quite la publicación del puerto 8080 del servicio `api` y levante dos réplicas con `docker compose up -d -scale api=2`. Compruebe con `docker compose ps` que hay dos contenedores del API y explique: (a) cómo reparte Docker las solicitudes de `biblio-web` entre ellos; (b) por qué el control de concurrencia optimista sigue funcionando con dos réplicas; y (c) qué pasaría con el *pool* de conexiones a PostgreSQL si se levantaran veinte réplicas.

CAPÍTULO 9

Pruebas de una aplicación distribuida en capas

Probar una aplicación distribuida es más difícil que probar un programa de un solo proceso: hay que comprobar cada capa por separado, las capas juntas, la comunicación entre niveles y el comportamiento ante accesos simultáneos. La literatura sobre pruebas de sistemas basados en microservicios y en API REST coincide en que ninguna técnica cubre todo por sí sola y en que hacen falta varios niveles de prueba complementarios [30], [31]. La separación en capas ayuda: cada capa se puede probar sola reemplazando a las demás.

9.1 Niveles de prueba en BiblioUTEQ

Tabla 9.1: Pruebas automatizadas del proyecto

Nivel	Clase de prueba	Qué se reemplaza y qué se comprueba	Casos
Unitaria	`PrestamoServicioTest`	Repositorios simulados con Mockito; reglas del negocio y cálculo de la multa	5
Capa de servicios web	`PrestamoControladorTest`	Servicio simulado; códigos HTTP y formato de error	4
Capa de acceso a datos	`PrestamoRepositorioTest`	Nada: PostgreSQL 18 real en un contenedor; migraciones, consultas y restricciones	3
Integración del API	`BiblioApiIntegracionTest`	Nada: servidor HTTP real y PostgreSQL 18; ciclo completo y concurrencia	3
Capa de presentación	`CatalogoWebControladorTest`	Cliente del API simulado; vistas y manejo de la caída del API	2
		Total	**17**

Todas se ejecutan con un solo comando, que es el mismo que usa la integración continua:

```
.\mvnw.cmd verify
```

En la verificación de esta guía, las 17 pruebas pasaron con JDK 27 y con JDK 21. Las que usan PostgreSQL necesitan Docker Desktop en ejecución; si Docker no está disponible, se omiten en lugar de fallar, gracias a la anotación `@Testcontainers` con el atributo `disabledWithoutDocker` activado.

9.2 Pruebas unitarias de la capa de negocio

Una prueba unitaria comprueba una clase aislada. Para probar `PrestamoServicio` sin base de datos, sus dependencias se reemplazan por objetos simulados (*mocks*) con Mockito: se les indica qué devolver y después se verifica cómo se usaron. Esto es posible precisamente porque el servicio recibe sus dependencias por el constructor (capítulo 2).

Listado 9.1: Prueba unitaria del servicio de préstamos con reloj fijo y repositorios simulados

```

    @BeforeEach
    void preparar() {
        Clock reloj = Clock.fixed(HOY.atStartOfDay(ZONA).toInstant(), ZONA);
        ReglasPrestamo reglas = new ReglasPrestamo(3, 7, new BigDecimal("0.25"));
        servicio = new PrestamoServicio(prestamos, libros, socios, reglas, reloj, eventos
↪ );
        socio = new Socio("E-0001", "Ana Torres", "ana.torres@uteq.edu.ec");
        libro = new Libro("978-1-4920-3402-5", "Building Microservices", "S. Newman", (
↪ short) 2021, 1);
    }

    @Test
    void registraPrestamoRetiraEjemplarYPublicaEvento() {
        when(socios.findById(1L)).thenReturn(Optional.of(socio));
        when(prestamos.countBySocioIdAndEstado(1L, EstadoPrestamo.ACTIVO)).thenReturn(0L);
↪
        when(libros.findById(10L)).thenReturn(Optional.of(libro));
        when(prestamos.save(any(Prestamo.class))).thenAnswer(inv -> inv.getArgument(0));

        Prestamo p = servicio.registrar(1L, 10L);

        assertThat(p.getEstado()).isEqualTo(EstadoPrestamo.ACTIVO);
        assertThat(p.getFechaVencimiento()).isEqualTo(HOY.plusDays(7));
        assertThat(libro.getEjemplaresDisponibles()).isZero();
        verify(eventos).publishEvent(any(PrestamoRegistradoEvento.class));
    }

    @Test
    void rechazaCuandoElSocioAlcanzoElLimite() {
        when(socios.findById(1L)).thenReturn(Optional.of(socio));
        when(prestamos.countBySocioIdAndEstado(1L, EstadoPrestamo.ACTIVO)).thenReturn(3L);
↪

        assertThatThrownBy(() -> servicio.registrar(1L, 10L))
                .isInstanceOf(ReglaNegocioException.class)
                .extracting("codigo").isEqualTo("LIMITE_PRESTAMOS");
        verify(prestamos, never()).save(any());
    }
```

El reloj fijo hace que la prueba sea repetible: el préstamo vence siempre siete días después del 1 de febrero de 2027, cualquiera que sea el día en que se ejecute.

9.3 Pruebas de la capa de servicios web

La anotación `@WebMvcTest` carga solo la capa web (el controlador y el manejador de errores) y reemplaza el servicio por un simulado. Así se comprueba el contrato HTTP sin tocar la base de datos: códigos de estado, formato Problem Details y campos de la respuesta.

Listado 9.2: Prueba de la capa web con `MockMvcTester`

```java
/** Prueba de la capa de servicios web aislada: solo se cargan el controlador y el
  ↪ manejador de errores. */
@WebMvcTest(PrestamoControlador.class)
class PrestamoControladorTest {

    @Autowired
    private MockMvcTester mvc;

    @MockitoBean
    private PrestamoServicio servicio;

    private static final String CUERPO = """
            {"socioId": 1, "libroId": 10}
            """;

    @Test
    void registrarDevuelve201ConElPrestamo() {
        Libro libro = new Libro("978-1-4920-3402-5", "Building Microservices", "S. Newman
  ↪ ", (short) 2021, 1);
        Socio socio = new Socio("E-0001", "Ana Torres", "ana.torres@uteq.edu.ec");
        Prestamo prestamo = Prestamo.registrar(libro, socio, LocalDate.of(2027, 2, 1), 7);
  ↪
        when(servicio.registrar(1L, 10L)).thenReturn(prestamo);

        assertThat(mvc.post().uri("/api/v1/prestamos").contentType(MediaType.
  ↪ APPLICATION_JSON).content(CUERPO))
                .hasStatus(HttpStatus.CREATED)
                .bodyJson().extractingPath("$.estado").isEqualTo("ACTIVO");
    }

    @Test
    void cuerpoInvalidoDevuelve400EnFormatoProblemDetails() {
        assertThat(mvc.post().uri("/api/v1/prestamos").contentType(MediaType.
  ↪ APPLICATION_JSON)
                .content("{\"socioId\": -5}"))
                .hasStatus(HttpStatus.BAD_REQUEST)
                .hasContentType(MediaType.APPLICATION_PROBLEM_JSON);
    }

    @Test
    void reglaDeNegocioDevuelve422ConSuCodigo() {
        when(servicio.registrar(1L, 10L))
                .thenThrow(new ReglaNegocioException("SIN_EJEMPLARES", "No hay ejemplares
  ↪ "));

        assertThat(mvc.post().uri("/api/v1/prestamos").contentType(MediaType.
  ↪ APPLICATION_JSON).content(CUERPO))
                .hasStatus(HttpStatus.UNPROCESSABLE_CONTENT)
                .bodyJson().extractingPath("$.codigo").isEqualTo("SIN_EJEMPLARES");
    }
```

9.4 Pruebas con una base de datos real: Testcontainers

Probar el acceso a datos contra una base de datos distinta de la de producción (por ejemplo, una base en memoria) deja pasar errores: las restricciones `CHECK`, los índices parciales o las particularidades de SQL de PostgreSQL no se comportan igual. Testcontainers resuelve esto levantando, para cada ejecución de las pruebas, un contenedor real de PostgreSQL 18 que

se destruye al terminar. La anotación `@ServiceConnection` configura automáticamente la conexión de Spring hacia ese contenedor.

Listado 9.3: Configuración compartida del contenedor de PostgreSQL 18 para las pruebas

```java
/**
 * Levanta un PostgreSQL 18 real en Docker para las pruebas. @ServiceConnection
 * configura el DataSource de Spring con la URL, el usuario y la clave del contenedor.
 */
@TestConfiguration(proxyBeanMethods = false)
public class ContenedoresPrueba {

    @Bean
    @ServiceConnection
    PostgreSQLContainer postgres() {
        return new PostgreSQLContainer("postgres:18");
    }
}
```

Listado 9.4: Prueba de la capa de acceso a datos contra PostgreSQL 18

```java
/**
 * Prueba de la capa de acceso a datos contra PostgreSQL 18 real: valida el esquema
 * creado por Flyway, las consultas JPQL y las restricciones CHECK de la base de datos.
 */
@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@Import(ContenedoresPrueba.class)
@Testcontainers(disabledWithoutDocker = true)
class PrestamoRepositorioTest {

    @Autowired
    private PrestamoRepositorio prestamos;
    @Autowired
    private LibroRepositorio libros;
    @Autowired
    private SocioRepositorio socios;
    @Autowired
    private JdbcTemplate jdbc;

    @Test
    void flywayCargaLosDatosIniciales() {
        assertThat(libros.count()).isEqualTo(4);
        assertThat(socios.count()).isEqualTo(3);
    }

    @Test
    void cuentaPrestamosActivosYTraeElDetalleEnUnaConsulta() {
        Socio socio = socios.findAllByOrderByNombres().getFirst();
        Libro libro = libros.findByIsbn("978-90-815406-3-6").orElseThrow();
        prestamos.save(Prestamo.registrar(libro, socio, LocalDate.of(2027, 2, 1), 7));

        assertThat(prestamos.countBySocioIdAndEstado(socio.getId(), EstadoPrestamo.ACTIVO
    )).isEqualTo(1);
        List<Prestamo> lista = prestamos.buscarPorSocio(socio.getId());
        assertThat(lista).hasSize(1);
        assertThat(lista.getFirst().getLibro().getTitulo()).isEqualTo("Distributed
    Systems");
    }

    @Test
    void laBaseDeDatosRechazaEjemplaresNegativosAunqueElCodigoFalle() {
        // Defensa en profundidad: la regla tambien esta en la base de datos (CHECK).
```

```
        assertThatThrownBy(() -> jdbc.update("UPDATE libro SET ejemplares_disponibles =
↪ -1 WHERE isbn = ?",
                "978-1-4920-3402-5"))
                .isInstanceOf(DataIntegrityViolationException.class);
    }
}
```

La última prueba del listado 9.4 intenta saltarse el código Java y escribir directamente un valor negativo en la base de datos: la restricción `CHECK` de la migración V1 lo impide. Es la comprobación de la defensa en profundidad descrita en el capítulo 4.

9.5 Prueba de integración con concurrencia

La prueba de integración levanta el API completo en un puerto aleatorio, con su servidor HTTP real, y le envía solicitudes como lo haría `biblio-web`. La más interesante para un sistema distribuido es la del listado 9.5: crea un libro con un solo ejemplar y lanza tres solicitudes de préstamo en el mismo instante, desde tres hilos que esperan una misma señal de salida.

Listado 9.5: Tres solicitudes simultáneas por el único ejemplar de un libro

```
    @Test
    void solicitudesSimultaneasNoPrestanMasEjemplaresDeLosQueExisten() throws Exception {
        LibroRespuesta libro = crearLibro("978-0-00-000002-8", 1);
        int clientes = 3; // los tres socios de los datos iniciales piden el unico
↪ ejemplar a la vez
        CountDownLatch salida = new CountDownLatch(1);
        List<Future<HttpStatus>> resultados = new ArrayList<>();

        try (ExecutorService hilos = Executors.newFixedThreadPool(clientes)) {
            for (long socio = 1; socio <= clientes; socio++) {
                long socioId = socio;
                Callable<HttpStatus> tarea = () -> {
                    salida.await(); // todos parten en el mismo instante
                    return prestar(socioId, libro.id());
                };
                resultados.add(hilos.submit(tarea));
            }
            salida.countDown();
        }

        long exitosos = 0;
        for (Future<HttpStatus> r : resultados) {
            HttpStatus estado = r.get();
            assertThat(estado).isIn(HttpStatus.CREATED, HttpStatus.CONFLICT, HttpStatus.
↪ UNPROCESSABLE_CONTENT);
            if (estado == HttpStatus.CREATED) {
                exitosos++;
            }
        }
        assertThat(exitosos).isEqualTo(1);
        cliente.get().uri("/api/v1/libros/{id}", libro.id())
                .exchange()
                .expectBody().jsonPath("$.ejemplaresDisponibles").isEqualTo(0);
    }
```

La prueba acepta que las solicitudes perdedoras terminen con 409 (perdieron la carrera en el control optimista) o con 422 (llegaron cuando el ejemplar ya estaba prestado); lo que no acepta es más de un préstamo exitoso ni un número de ejemplares disponibles distinto de cero.

Esta es la forma de probar una propiedad de seguridad de un sistema concurrente: no se fija el orden de los eventos, que no se puede controlar, sino la invariante que debe cumplirse en cualquier orden.

9.6 Prueba de la capa de presentación

Listado 9.6: Prueba de la capa de presentación con el API simulado

```
/** La capa de presentacion se prueba sin el API: el cliente HTTP se simula. */
@WebMvcTest({ CatalogoWebControlador.class, ManejadorFallosApi.class })
class CatalogoWebControladorTest {

    @Autowired
    private MockMvcTester mvc;

    @MockitoBean
    private BiblioApiCliente api;

    @Test
    void muestraLosLibrosQueDevuelveElApi() {
        when(api.listarLibros(null)).thenReturn(List.of(
                new LibroDatos(1L, "978-90-815406-3-6", "Distributed Systems", "M. van
  ↪ Steen", (short) 2023, 3, 0)));

        assertThat(mvc.get().uri("/"))
                .hasStatusOk()
                .hasViewName("catalogo")
                .bodyText().contains("Distributed Systems").contains("0 de 3");
    }

    @Test
    void siElApiNoRespondeMuestraUnaPaginaControlada() {
        when(api.listarLibros(null)).thenThrow(new ResourceAccessException("Connection
  ↪ refused"));

        assertThat(mvc.get().uri("/"))
                .hasStatus(HttpStatus.SERVICE_UNAVAILABLE)
                .hasViewName("servicio-no-disponible");
    }
}
```

La segunda prueba del listado 9.6 simula que el API no responde y comprueba que el usuario recibe la página controlada con el código 503, que es el comportamiento diseñado en el capítulo 7.

Práctica 9.1: medir la cobertura y probar la carga

(a) Agregue el complemento JaCoCo al `pom.xml` de `biblio-api`, ejecute `.\mvnw.cmd verify` y abra el informe `target/site/jacoco/index.html`. Identifique qué ramas de `ManejadorErrores` y de `Prestamo` no están cubiertas y escriba las pruebas que faltan. (b) Con una herramienta de pruebas de carga de su elección, envíe 50 solicitudes por segundo durante un minuto a `GET /api/v1/libros` con el sistema en Docker Compose y reporte la latencia en los percentiles 50, 95 y 99. Repita la medición cinco veces y reporte la media y la dispersión.

CAPÍTULO 10

Calidad, integración continua y observabilidad

Una aplicación que funciona en el equipo del desarrollador todavía no es una aplicación de calidad. Este capítulo relaciona lo construido con el modelo de calidad ISO/IEC 25010, automatiza su verificación en cada cambio con integración continua y muestra cómo observar el sistema mientras se ejecuta.

10.1 El modelo de calidad ISO/IEC 25010

La norma ISO/IEC 25010:2023 define el modelo de calidad de un producto de software como un conjunto de características que se pueden especificar, medir y evaluar [1]. La tabla 10.1 relaciona cada característica con decisiones concretas de BiblioUTEQ y con la forma de evidenciarlas. En la práctica, cuando su equipo afirme que el PFC cumple una característica, debe poder señalar la evidencia de la última columna.

Tabla 10.1: Características de ISO/IEC 25010:2023 en BiblioUTEQ

Característica	Decisión en el proyecto	Evidencia verificable
Adecuación funcional	Reglas R1 a R6 implementadas en dominio y negocio	Pruebas unitarias e integración (capítulo 9)
Eficiencia de desempeño	Índice parcial, `join fetch`, *pool* de conexiones	Latencias medidas en la práctica 9.1
Compatibilidad	Contrato OpenAPI y errores RFC 9457	`/v3/api-docs` y pruebas de la capa web
Capacidad de interacción	Mensajes de error comprensibles para el usuario	Figura 7.1
Fiabilidad	Tiempos de espera, página de servicio no disponible, control optimista	Falla provocada del capítulo 8; prueba de concurrencia
Seguridad	Base de datos no expuesta al usuario, contenedores sin privilegios, secretos por variables de entorno	`compose.yaml` y `Dockerfile`
Mantenibilidad	Capas con dependencias en un solo sentido, inyección de dependencias, pruebas por capa	Estructura de paquetes y 17 pruebas automatizadas
Flexibilidad	Configuración externa, imágenes de contenedor, JDK intercambiable	Mismo artefacto en IntelliJ IDEA y en Docker

Característica	Decisión en el proyecto	Evidencia verificable
Protección (*safety*)	Restricciones `CHECK` que impiden estados imposibles	Prueba de la restricción en `PrestamoRepositorioTest`

10.2 Integración continua con GitHub Actions

La integración continua consiste en compilar y probar el sistema automáticamente cada vez que alguien sube cambios al repositorio, de modo que un error se detecte minutos después de introducirse y no días después. En arquitecturas distribuidas, la automatización de la construcción, las pruebas y el despliegue es una práctica central para poder cambiar cada parte con frecuencia y sin miedo [32].

Listado 10.1: Flujo de integración continua `.github/workflows/ci.yml`

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  construir-y-probar:
    runs-on: ubuntu-latest
    steps:
      - name: Obtener el codigo
        uses: actions/checkout@v7

      - name: Instalar JDK 27 (Temurin)
        uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: '27'
          cache: maven

      # Las pruebas con Testcontainers usan el Docker que ya trae el ejecutor ubuntu-latest
      - name: Compilar y ejecutar todas las pruebas
        run: ./mvnw -B verify
```

El flujo del listado 10.1 instala JDK 27 y ejecuta `mvnw verify`, el mismo comando que se usa en el equipo local. Las pruebas con Testcontainers funcionan en el ejecutor `ubuntu-latest` porque este ya trae Docker. Si alguna prueba falla, el *commit* queda marcado en rojo en GitHub y el equipo sabe exactamente qué cambio lo rompió.

Una regla de trabajo en equipo

Nadie integra cambios a la rama `main` si el flujo de integración continua está en rojo. En GitHub se puede imponer con una regla de protección de la rama que exija que el trabajo `construir-y-probar` termine en verde antes de aceptar una solicitud de integración (*pull request*).

10.3 Observabilidad

Cuando un sistema distribuido falla en producción, no se puede depurar paso a paso: solo se puede observar lo que el sistema registra sobre sí mismo. La **observabilidad** es esa capacidad de entender el estado interno del sistema a partir de sus registros (*logs*), sus métricas y sus trazas. Un estudio con profesionales de la industria identificó la observabilidad como una necesidad creciente en los sistemas distribuidos y documentó las dificultades prácticas para lograrla [33].

BiblioUTEQ incluye Spring Boot Actuator, que publica el estado de salud del servicio y sus métricas sin escribir código adicional:

Listado 10.2: Salud y una métrica del API en ejecución

```
GET http://localhost:8080/actuator/health
{"groups":["liveness","readiness"],"status":"UP"}

GET http://localhost:8080/actuator/metrics/http.server.requests
measurements: COUNT, TOTAL_TIME, MAX
availableTags: exception, method, error, uri, outcome, status
```

- **Salud.** Los grupos `liveness` y `readiness` permiten que un orquestador sepa si debe reiniciar el contenedor o si todavía no debe enviarle tráfico.
- **Métricas.** La métrica `http.server.requests` cuenta las solicitudes y mide su duración, separadas por ruta, método y código de estado. Con ella se calcula la tasa de errores o la latencia de cada operación.
- **Registros de negocio.** El observador `NotificadorPrestamos` escribe una línea por cada préstamo confirmado, con su identificador, que permite reconstruir qué pasó.

Práctica 10.1: métricas en Prometheus y Grafana

Agregue a `biblio-api` la dependencia `micrometer-registry-prometheus`, exponga el punto `/actuator/prometheus` y agregue al `compose.yaml` un contenedor de Prometheus que lo consulte cada 15 segundos y uno de Grafana. Construya un panel con la tasa de solicitudes por ruta, la latencia en los percentiles 50, 95 y 99, y la tasa de respuestas 409 y 422. Agregue además un contador propio, `biblio.prestamos.registrados`, incrementado desde el observador de préstamos.

CAPÍTULO 11

Aplicación al Proyecto Fin de Curso

Este capítulo traslada lo aprendido con BiblioUTEQ al Proyecto Fin de Curso (PFC) de cada equipo. Las actividades siguen el orden de la Unidad 5: primero se revisa la arquitectura, luego se refactoriza y por último se verifica la calidad.

11.1 Señales de que las capas se están erosionando

Antes de modificar el PFC, revise si presenta alguno de estos problemas. Cada uno es una forma concreta de la erosión de la arquitectura descrita en el capítulo 1 [7].

Tabla 11.1: Señales de erosión de la arquitectura en capas

Señal	Por qué es un problema	Cómo se corrige
El controlador usa un repositorio directamente	Se salta la capa de negocio; las reglas quedan dispersas	Mover la lógica a un servicio
El API devuelve entidades JPA	Expone datos internos y ata el contrato al esquema	DTO de respuesta (capítulo 6)
Reglas del negocio en el controlador o en la vista	La misma regla se repite y se contradice entre clientes	Llevarla al dominio o al servicio
`@Transactional` en el controlador	La transacción incluye trabajo de presentación y dura más de lo necesario	Delimitar la transacción en el servicio
Servicios que solo reenvían al repositorio	Capas que no aportan nada; las solicitudes atraviesan capas vacías [3]	Aceptarlo si es poco frecuente; si es la mayoría, revisar el diseño
Llamadas remotas sin tiempo de espera	Una falla en otro nivel bloquea al que llama	Configurar tiempos de espera (capítulo 7)
Excepciones técnicas que llegan al cliente	El cliente recibe trazas internas y no puede reaccionar	Manejador global con Problem Details

11.2 Actividades

Actividad 11.1: diagnóstico de capas del PFC

Para el microservicio principal de su PFC, dibuje sus capas y las dependencias reales entre sus paquetes (puede usar el diagrama de dependencias de IntelliJ IDEA). Marque con rojo cada dependencia que viole la regla de la sección 1.3 y cada señal de la tabla 11.1 que encuentre. Entregue el diagrama y la lista priorizada de correcciones en el documento del PFC.

Actividad 11.2: reglas de capas como pruebas

Agregue la biblioteca ArchUnit a las pruebas del PFC y escriba al menos tres reglas que fallen si alguien viola la arquitectura: (a) las clases del paquete de negocio no dependen del paquete web; (b) los controladores no dependen de los repositorios; (c) las entidades no se usan en las firmas públicas de los controladores. Ejecútelas en la integración continua: a partir de ese momento, la erosión se detecta automáticamente.

Actividad 11.3: refactorización guiada

Corrija las tres violaciones de mayor prioridad de la actividad 11.1. Por cada una, haga un *commit* independiente cuyo mensaje explique la regla que se restablece, y verifique que las pruebas existentes siguen pasando antes y después del cambio.

Actividad 11.4: matriz de calidad del PFC

Construya para su PFC una tabla como la tabla 10.1: para cada característica de ISO/IEC 25010:2023 que su equipo priorice, indique la decisión de diseño que la favorece y la evidencia verificable en el repositorio (prueba, medición, configuración o captura). No incluya características sin evidencia.

11.3 Lista de verificación de una aplicación distribuida en capas

Antes de entregar, compruebe que su sistema cumple

- ☐ Cada capa usa solo a la inmediatamente inferior; ninguna clase de negocio importa clases web.
- ☐ Los controladores no contienen reglas del negocio y no usan repositorios.
- ☐ El API usa DTO, valida su entrada y responde errores con Problem Details.
- ☐ Las transacciones se delimitan en la capa de negocio.
- ☐ Los datos que pueden modificarse a la vez tienen control de concurrencia y una prueba que lo demuestra.
- ☐ El esquema de la base de datos se crea con migraciones versionadas.
- ☐ Toda llamada entre niveles tiene tiempo de espera y un manejo controlado de la falla.
- ☐ La configuración que cambia entre ambientes (URL, usuarios, claves) se lee de variables de entorno.
- ☐ El sistema completo se levanta con `docker compose up` desde un clon limpio.
- ☐ Existen pruebas por capa y de integración, y la integración continua está en verde.
- ☐ El servicio publica su salud y sus métricas.

APÉNDICE A

Estructura completa del proyecto

Listado A.1: Archivos del proyecto BiblioUTEQ

```
biblio/
|-- pom.xml                      (proyecto padre: Spring Boot 4.1.2, Java 27, dos modulos
  ↪ )
|-- mvnw, mvnw.cmd, .mvn/        (Maven Wrapper: Maven 3.9.16)
|-- compose.yaml                 (tres niveles: web, api, db)
|-- README.md
|-- .github/workflows/ci.yml     (integracion continua)
|-- biblio-api/                  (nivel 2: servicios web, negocio, dominio y acceso a
  ↪ datos)
|   |-- Dockerfile
|   |-- pom.xml
|   `-- src/
|       |-- main/java/ec/edu/uteq/biblio/api/
|       |   |-- BiblioApiApplication.java
|       |   |-- config/          ConfiguracionReloj, ConfiguracionOpenApi
|       |   |-- web/             LibroControlador, SocioControlador, PrestamoControlador,
|       |   |                    ManejadorErrores
|       |   |-- web/dto/         LibroSolicitud, LibroRespuesta, SocioRespuesta,
|       |   |                    PrestamoSolicitud, PrestamoRespuesta
|       |   |-- negocio/         CatalogoServicio, PrestamoServicio, ReglasPrestamo,
|       |   |                    NotificadorPrestamos, RecursoNoEncontradoException
|       |   |-- dominio/         Libro, Socio, Prestamo, EstadoPrestamo,
|       |   |                    PrestamoRegistradoEvento, ReglaNegocioException
|       |   `-- datos/           LibroRepositorio, SocioRepositorio, PrestamoRepositorio
|       |-- main/resources/
|       |   |-- application.yaml
|       |   `-- db/migration/    V1__esquema_inicial.sql, V2__datos_iniciales.sql
|       `-- test/java/ec/edu/uteq/biblio/api/
|           |-- ContenedoresPrueba.java
|           |-- negocio/         PrestamoServicioTest
|           |-- web/             PrestamoControladorTest
|           |-- datos/           PrestamoRepositorioTest
|           `-- integracion/     BiblioApiIntegracionTest
`-- biblio-web/                  (nivel 1: presentacion)
    |-- Dockerfile
    |-- pom.xml
    `-- src/
        |-- main/java/ec/edu/uteq/biblio/web/
        |   |-- BiblioWebApplication.java
        |   |-- cliente/         BiblioApiCliente, Modelos
        |   |-- config/          ClienteApiConfig
        |   `-- controlador/     CatalogoWebControlador, PrestamoWebControlador,
        |                        ManejadorFallosApi
        |-- main/resources/
        |   |-- application.yaml
        |   |-- templates/       fragmentos, catalogo, prestamos, servicio-no-disponible
        |   `-- static/css/      estilos.css
        `-- test/java/ec/edu/uteq/biblio/web/
            `-- CatalogoWebControladorTest
```

APÉNDICE B

Comandos de referencia

Tabla B.1: Comandos frecuentes en PowerShell

Comando	Para qué sirve
`.\mvnw.cmd verify`	Compilar y ejecutar todas las pruebas
`.\mvnw.cmd -DskipTests package`	Generar los `.jar` sin ejecutar pruebas
`.\mvnw.cmd -pl biblio-api test`	Ejecutar solo las pruebas del API
`docker compose up -d db`	Levantar solo PostgreSQL para trabajar desde IntelliJ IDEA
`docker compose up -d -build`	Construir las imágenes y levantar los tres niveles
`docker compose ps`	Ver el estado de los contenedores
`docker compose logs -f api`	Seguir los registros del API
`docker compose stop api`	Detener un nivel para provocar una falla parcial
`docker compose down`	Detener y eliminar los contenedores (conserva los datos)
`docker compose down -v`	Eliminar también el volumen de datos
`docker compose exec db psql -U biblio -d biblio`	Abrir una consola SQL en PostgreSQL
`Invoke-RestMethod http://localhost:8080/api/v1/libros`	Consultar el API desde PowerShell

Referencias

[1] International Organization for Standardization, *ISO/IEC 25010:2023 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*, ISO, Geneva, Switzerland, 2023.

[2] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002, ISBN: 978-0-321-12742-6.

[3] M. Richards y N. Ford, *Fundamentals of Software Architecture, A Modern Engineering Approach*, 2.ª ed. O'Reilly Media, 2025, ISBN: 978-1-098-17551-1.

[4] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.ª ed. Amsterdam, The Netherlands: Maarten van Steen, 2023, ISBN: 978-90-815406-3-6. dirección: `https://www.distributed-systems.net/`.

[5] M. Kleppmann y C. Riccomini, *Designing Data-Intensive Applications*, 2.ª ed. Sebastopol, CA, USA: O'Reilly Media, 2026, ISBN: 978-1-098-11906-5.

[6] L. Bass, P. Clements y R. Kazman, *Software Architecture in Practice*, 4.ª ed. Addison-Wesley, 2021, ISBN: 978-0-13-688609-9.

[7] R. Li, P. Liang, M. Soliman y P. Avgeriou, «Understanding Software Architecture Erosion: A Systematic Mapping Study,» *Journal of Software: Evolution and Process*, vol. 34, n.º 3, e2423, 2022. DOI: `10.1002/smr.2423`.

[8] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2.ª ed. Sebastopol, CA, USA: O'Reilly Media, 2021, ISBN: 978-1-4920-3402-5.

[9] G. Blinowski, A. Ojdowska y A. Przybylek, «Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,» *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: `10.1109/ACCESS.2022.3152803`.

[10] R. C. Martin, *Clean Architecture, A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017, ISBN: 978-0-13-449416-6.

[11] R. Laigner, D. Mendonça, A. Garcia y M. Kalinowski, «Cataloging Dependency Injection Anti-Patterns in Software Systems,» *Journal of Systems and Software*, vol. 184, pág. 111 125, 2022. DOI: `10.1016/j.jss.2021.111125`.

[12] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns, Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994, ISBN: 978-0-201-63361-0.

[13] E. Evans, *Domain-Driven Design, Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003, ISBN: 978-0-321-12521-7.

[14] JetBrains. «IntelliJ IDEA as a Unified Product.» (2025), dirección: `https://www.jetbrains.com/help/idea/intellij-idea-single-distribution.html` (visitado 26-10-2026).

[15] JetBrains. «What's New in IntelliJ IDEA 2026.2.» (2026), dirección: `https://blog.jetbrains.com/idea/2026/07/intellij-idea-2026-2/` (visitado 26-10-2026).

[16] Oracle. «The Arrival of Java 27!» (15 de sep. de 2026), dirección: `https://inside.java/2026/09/15/jdk-27-available/` (visitado 26-10-2026).

[17] Eclipse Adoptium. «Available Releases (API v3).» (2026), dirección: `https://api.adoptium.net/v3/info/available_releases` (visitado 26-10-2026).

[18] Spring. «Spring Boot 4.1.2 Available Now.» (23 de oct. de 2026), dirección: https://spring.io/blog/2026/10/23/spring-boot-4-1-2-available-now/ (visitado 26-10-2026).

[19] Spring. «Spring Boot Reference: System Requirements.» (2026), dirección: https://docs.spring.io/spring-boot/system-requirements.html (visitado 26-10-2026).

[20] PostgreSQL Global Development Group. «PostgreSQL: The World's Most Advanced Open Source Relational Database.» (2026), dirección: https://www.postgresql.org/ (visitado 26-10-2026).

[21] PostgreSQL Global Development Group. «PostgreSQL 19 RC 1 Released!» (15 de oct. de 2026), dirección: https://www.postgresql.org/about/news/postgresql-19-rc-1-released-3409/ (visitado 26-10-2026).

[22] Docker. «Docker Desktop Release Notes.» (2026), dirección: https://docs.docker.com/desktop/release-notes/ (visitado 26-10-2026).

[23] H. T. Kung y J. T. Robinson, «On Optimistic Methods for Concurrency Control,» *ACM Transactions on Database Systems*, vol. 6, n.º 2, págs. 213-226, 1981. DOI: 10.1145/319566.319567.

[24] R. T. Fielding, «Architectural Styles and the Design of Network-based Software Architectures,» Tesis doct., University of California, Irvine, Irvine, CA, USA, 2000.

[25] R. T. Fielding, M. Nottingham y J. Reschke, «HTTP Semantics,» RFC Editor, RFC 9110, jun. de 2022. DOI: 10.17487/RFC9110. dirección: https://www.rfc-editor.org/rfc/rfc9110.

[26] J. Bogner, S. Kotstein y T. Pfaff, «Do RESTful API Design Rules Have an Impact on the Understandability of Web APIs?» *Empirical Software Engineering*, vol. 28, n.º 6, pág. 132, 2023. DOI: 10.1007/s10664-023-10367-y.

[27] A. Lercher, J. Glock, C. Macho y M. Pinzger, «Microservice API Evolution in Practice: A Study on Strategies and Challenges,» *Journal of Systems and Software*, vol. 215, pág. 112 110, 2024. DOI: 10.1016/j.jss.2024.112110.

[28] M. Nottingham, E. Wilde y S. Dalal, «Problem Details for HTTP APIs,» RFC Editor, RFC 9457, jul. de 2023. DOI: 10.17487/RFC9457. dirección: https://www.rfc-editor.org/rfc/rfc9457.

[29] OpenAPI Initiative, *OpenAPI Specification*, ver. 3.2.1, OpenAPI Initiative, The Linux Foundation, 10 de sep. de 2026. dirección: https://spec.openapis.org/oas/v3.2.1.html (visitado 26-10-2026).

[30] F. Ponce, R. Verdecchia, B. Miranda y J. Soldani, «Microservices Testing: A Systematic Literature Review,» *Information and Software Technology*, vol. 188, pág. 107 870, 2025. DOI: 10.1016/j.infsof.2025.107870.

[31] A. Golmohammadi, M. Zhang y A. Arcuri, «Testing RESTful APIs: A Survey,» *ACM Transactions on Software Engineering and Methodology*, vol. 33, n.º 1, págs. 1-41, 2023. DOI: 10.1145/3617175.

[32] M. Waseem, P. Liang y M. Shahin, «A Systematic Mapping Study on Microservices Architecture in DevOps,» *Journal of Systems and Software*, vol. 170, pág. 110 798, 2020. DOI: 10.1016/j.jss.2020.110798.

[33] S. Niedermaier, F. Koetter, A. Freymann y S. Wagner, «On Observability and Monitoring of Distributed Systems – An Industry Interview Study,» en *Service-Oriented Computing: 17th International Conference, ICSOC 2019, Toulouse, France, October 28–31, 2019, Proceedings*, S. Yangui, I. Bouassida Rodriguez, K. Drira y Z. Tari, eds., ép. Lecture Notes in Computer Science, vol. 11895, Cham: Springer, 2019, págs. 36-52. DOI: 10.1007/978-3-030-33702-5_3.